



University of Essex

Department of Computer Science
and Electronic Engineering

Privacy-Preserving Cross-User Memory for LLM-Based Web Agents via Differential Privacy

YASMINE JUNE FRIZLEN

250xx

**A thesis submitted for the degree of Master in Science:
Artificial Intelligence**

Supervisor: Professor Luca Citi

August 17, 2026
Colchester

Preface

Contents

Preface	2
1 Introduction	3
2 Background	4
2.1 Related Work	4
2.1.1 Large Language Models (LLMs)	4
2.1.2 AI Agents	6
2.1.3 ReAct framework	7
2.1.4 Memory architecture	8
2.1.5 Memory creation and retrieval	10
2.1.6 Differential Privacy	12
2.2 Research question	15
2.2.1 Hypotheses	16
2.2.2 Scope	16
3 Project management	18
3.1 Agile	18
3.2 Tool use	19
3.2.1 Tuleap	19
3.2.2 Overleaf	20
3.2.3 GitLab and GitHub	20
3.2.4 Code quality tooling	20
3.2.5 LLMs	23
4 Methodology	25
4.1 System architecture	25
4.1.1 Agent	28
4.1.2 Private memory creation	28
4.1.3 Shared memory creation	28
4.2 Differential privacy mechanism	32
5 Implementation	37
5.1 Tech stack	37
5.1.1 Hardware	37
5.1.2 Software	38
5.2 Design choices	42
5.2.1 Agent	42
5.2.2 Private memory creation	45

5.2.3	Shared memory creation	47
6	Unanticipated difficulties and critical reflections	52
6.1	Unanticipated difficulties	52
6.1.1	Generating usable text with the Amin et al. mechanism	52
6.1.2	Hosting the WebArena benchmark	57
6.1.3	A low success rate on WebArena	57
6.1.4	Memory injection breaking the action grammar	58
6.1.5	Revising the hypotheses and experimental scenarios	59
6.2	Critical reflections	59
7	Results and Evaluation	64
7.0.1	Table of results	64
7.0.2	Agreement between the LLM judge and the WebArena harness	64
7.1	Discussion	64
8	Conclusion and future work	66
8.1	Conclusion	66
8.2	Lessons learnt	66
8.3	Limitations and future work	66
8.3.1	Limitations	66
8.3.2	Future work	68
9	Annex	69

List of Figures

2.1	Input transformation	5
2.2	Output distribution	5
2.3	The iterative execution cycle of the ReAct framework, combining internal LLM reasoning (<i>Thought</i>) and external environment interaction (<i>Action</i> and <i>Observation</i>), as introduced by Yao et al. ?.	7
3.1	Weekly Kanban board excerpt from Tuleap	20
3.2	GitLab merge request template	21
4.1	System architecture	27
4.2	Private memory mechanism	29
4.3	Shared memory generation architecture	29
4.4	Differential privacy mechanism	33
5.1	System instructions for extracting memory items from agent trajectories: the left panel targets successful trajectories (summarizing why they succeed), while the right targets failed trajectories (reflecting on failure and deriving lessons).	48
6.1	Comparison of Gemma 2 and Gemma 2 IT model	53
6.2	View of first two memories generated with the Amin et Al. mechanism using the TREC dataset.	54
6.3	Token logit heights	55
9.1	"Decision-curator" command	69
9.2	"Thesis-task-curator" command	71
9.3	"Thesis-tasks" skill	73
9.4	"Editing academic text" skill	75
9.5	Coding standards	83

List of Tables

2.1	Memory architectures for language agents, by the memory operation each one addresses and the scope of the resulting store.	10
2.2	Frameworks for creating episodic memories, by what each one writes to the store and how far that entry transfers.	11
5.1	Action Space of WebArena	45
5.2	System instruction and input prompt template used by the LLM judge to evaluate web-navigation agent trajectories.	46
5.3	Interim. Differential privacy hyperparameters of the pilot configuration, applied identically to Step 1 and Step 3. These values are provisional and are superseded once the budget is retuned.	49
5.4	Step 1 - System prompt used by the Gemma model to generate the differentially private labels for each cycle.	50
5.5	Step 3 - System prompt used by the Gemma model to generate the differentially private content for each label bucket.	51
5.6	Step 4 - System prompt used by the Qwen model to generate a title and description for each synthesised shared-memory item.	51
6.1	The ten toy memories (<code>scripts/amin_et_al/examples.csv</code>) used as the private corpus for the diagnostics in Section 6.1.1.	53
6.2	Round-1 label prompts compared on the toy fixture at a realised $(\epsilon, \delta) = (26.2, 0.01)$, $k = 3$, $r = 80$. Every variant returns a JSON array of strings, is parsed by the same three-stage parser, and spends the same private-token budget; only the instruction prose differs. <code>json_discriminative</code> was promoted into the pipeline.	61
6.3	Labels produced by the two generation modes on the toy fixture at a realised $(\epsilon, \delta) = (26.2, 0.01)$, $k = 3$, $r = 80$	62
6.4	Six bucketing methods on the same 100 memories at $k = 4$. The mechanism as deployed ran at a realised ϵ of 10.9 with $\delta = 0.01$ and $r = 80$. BERTopic at its own defaults chose three clusters rather than four and assigned 65 of the 100 memories to an outlier bucket.	62
6.5	How the 753 failed trajectories of the no-memory pass-through ended.	62
6.6	The hypotheses as proposed and as revised. The revision changes the baseline of H1 from private-memory-only to no-memory, and replaces “a meaningful fraction” in H2 with a stated budget and an explicit failure condition.	63
6.7	The experimental scenarios as proposed and as revised. The proposal simulated ten users inside one pass-through; the revision runs the same 812 tasks twice and varies which memory store the second run may read.	63

7.1	Interim. Agreement between the LLM-as-judge verdict and the WebArena harness reward across the 812 baseline trajectories.	65
-----	--	----

Abstract

xyz
xyz
xyz

Keywords: xyz · xyz · xyz · xyz · xyz · xyz · xyz · xyz · xyz · xyz

Introduction

<TBD> Add a text summarising what this project is about </TBD>
brief introduction of each concept

Background

This section discusses the most relevant related literature, and, on the basis of the gap this thesis has found, sets out the research question it aims to answer.

2.1 Related Work

This chapter provides the background and the literature review with a comprehensive overview of all relevant topics that will be built on in this thesis. The chapter first covers Large Language Models, AI Agents, the ReAct framework, memory architectures, memory creation and retrieval, differential privacy, and lastly, the research question resulting from the gap in the literature.

2.1.1 Large Language Models (LLMs)

Large Language Models form the backbone of today's AI agents. Their attention mechanism weighs the relevance of all input tokens against one another, which allows the model to capture long-range dependencies in text ? and has enabled tasks such as machine translation and question answering. The same capability underpins the reasoning and action selection required of a web-based agent, which is why LLMs are used in this thesis ?. Rather than provide a complete account of the transformer architecture underlying the LLM ?, this section will only detail the steps of the architecture that the methodology relies on. The three critical mechanisms are how the input is first transformed, how the output distribution is produced, and how one can steer the behaviour of the model at inference time. This section covers these three aspects in turn.

Input transformation

The models used in this work are decoder-only transformers, trained on a single objective of predicting the next token in a sequence ?. On the input side, a tokenizer splits the text into tokens, which are words or subwords drawn from a fixed vocabulary, typically constructed with byte-pair encoding ?. An embedding layer then maps each token to a learned numerical vector, and this sequence of vectors is what the transformer processes.

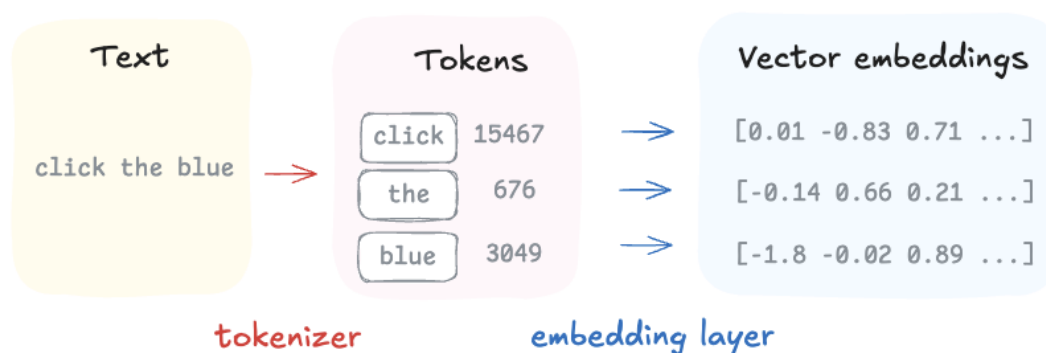


Figure 2.1: Input transformation

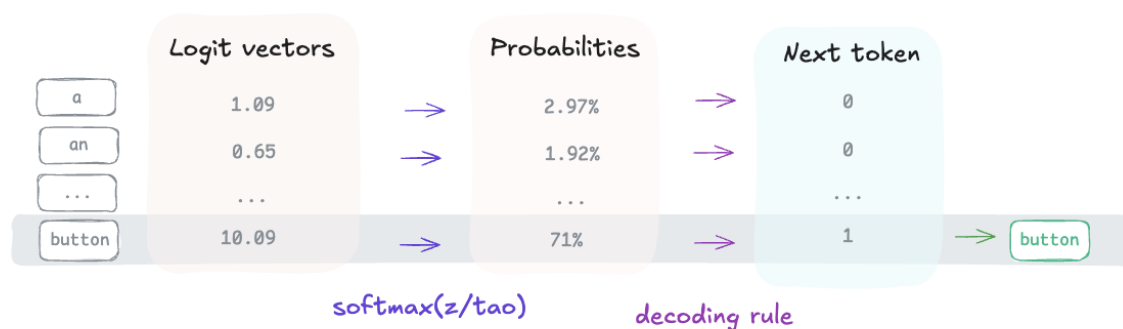


Figure 2.2: Output distribution

Output distribution

On the output side, the transformer produces a vector that contains one unnormalised score, called a logit, per vocabulary entry. A softmax function converts these logits into a probability distribution over the vocabulary, and a decoding rule selects one token from that distribution. The simplest rule is greedy decoding, which always takes the highest-probability token. Another rule is to sample the token from the probability distribution, so that a token with probability 0.9 is selected 90% of the time. This approach also allows the use of a temperature parameter. This parameter divides the logits before the softmax is applied. The effect of low temperatures (< 1) is a concentrated probability mass on the top-scoring tokens, while high temperatures (> 1) flatten the distribution. The selected token is appended to the input sequence, and generation repeats one token at a time until the model emits an end-of-sequence token or reaches a set token limit.

Steering model behaviour

The model's behaviour at inference time is steered through two mechanisms. First, the models used in this work are instruction-tuned, meaning further trained to follow natural-language instructions in a structured chat format, which requires arranging the input into a model-specific chat template. Second, the model's behaviour is steered by adding task descriptions, past actions, and retrieved memories to the prompt. This context can alter the output distribution without any change to the model's weights. This property is known as in-context learning, and it is what allows the agent's memories to influence behaviour without retraining. The limitation

of this property is that everything must fit within a fixed maximum number of tokens, which constrains how much observation and memory content the agent can be shown at each step.

Discussion

Since this thesis aims to equip modern, frontier webagents with novel memory capacity, using an LLM to steer the agent’s behaviour is the cleanest solution. It is in line with current implementations of frontier AI agents, such as those provided by OpenAI and Anthropic, which all rely on LLMs. An LLM-based agent’s ability to be adaptable to new situations, in contrast to rule-based agents, provides an additional benefit.

2.1.2 AI Agents

While the cultural relevance of AI agents has truly taken off in recent years, agents have existed for decades ?. In 1995, Wooldridge and Jennings ? characterised an agent by its autonomy, reactivity, pro-activeness, and social ability, while Ferber ? described an agent as an entity that can act, perceive its environment, and pursue goals. These definitions are still applicable today, and the latter definition is closely aligned with the framework this project uses for its agents. However, the technology available at the time constrained how those properties were realised. Symbolic rule engines and predefined knowledge bases could only respond to inputs their designers had anticipated ?. As a result of this limitation, agents faced persistent scalability and generalisation challenges when faced with novel inputs ?.

Reinforcement learning (RL) relaxed this constraint by enabling agents to learn policies through interaction with an environment rather than from hand-coded rules ?. Agents could develop competent behaviours through trial-and-error reward maximisation, without requiring a designer to enumerate every possible situation. The integration of RL with deep neural networks produced striking results in narrow domains, including superhuman performance in Atari games ? and the game of Go ?. However, these agents were trained for a specific task and could not transfer their capabilities across domains. An agent trained to play Go has no mechanism to navigate a website.

Large language models shifted the problem differently. Pre-trained on vast text corpora, an LLM generalises across language-based tasks without task-specific retraining ?. With an LLM as its basis, an AI agent wraps the model with access to external tools, function-calling capability, and the ability to reason sequentially across steps ?. The key addition of the LLM in comparison to earlier agents is that it can generalise, even over novel inputs. Wang et al. ? and Xi et al. ? document how this combination has since been deployed across web navigation, software engineering, information retrieval, and commercial task execution.

Agentic AI, as Sapkota et al. ? define it, extends this architecture by combining multiple agents that decompose goals, communicate with one another, and coordinate towards a shared objective. Whereas a single LLM-based agent reasons and acts in sequence, an agentic AI system distributes that reasoning across specialised agents, each responsible for a different aspect of the workflow. Commercial deployments such as OpenAI’s Operator illustrate this trajectory, completing multi-step web tasks through coordinated sub-task execution with minimal human oversight ?. For this system to work intelligently and over a long horizon, it requires some form of memory. Without persistent memory of past outcomes, each deployment begins from scratch, and each new user receives no benefit from the experience of those who came before. How experiential knowledge should be structured, stored and shared is the subject of this Section 2.1.4, and the privacy question it raises when shared between users is the central problem this thesis addresses.

Discussion

As highlighted in the previous section, LLM-based agents provide the baseline for the frontier of web-based agentic capabilities. This thesis aims to provide a solution to privacy-protecting memory for today’s leading technology, and thus bases itself on LLM-based agents.

2.1.3 ReAct framework

Before ReAct, there was no single standard architecture for LLM-based agents. The approaches that did exist each addressed a subpart of the problem ReAct would eventually solve: letting a language model reason about a task and act on an external environment within a single loop. Each built on the capabilities of a standalone LLM.

The first, Chain-of-Thought prompting, demonstrated that large language models improve their performance when prompted to generate intermediate reasoning steps ?. This reasoning, however, is a purely internal process. The model remains limited to the knowledge instantiated in its weights and can neither act on, nor receive information from, an external environment.

A second set of frameworks let language models generate action sequences for embodied tasks. Systems such as Language Models as Zero-Shot Planners ? and SayCan ? translated language-model knowledge into executable plans. Their major limitation was that plans were produced without feedback from the environment, so when the environment deviated from the model’s assumptions, the remaining plan could become invalid ?.

A third set of frameworks connected language models to external resources. WebGPT trained a model to search and navigate a text-based browser ?, while MRKL Systems proposed routing language-model requests to specialised knowledge and reasoning modules ?. These systems included components later used in agent architectures, namely browsing, tool invocation, and environment interaction, but relied on task-specific training or modular routing rather than a generic prompting loop, and did not model reasoning explicitly.

Yao et al. ? combined these capabilities in the ReAct framework, which interleaves reasoning (*Thought*), environment interaction (*Action*), and feedback from the environment (*Observation*) in a single trajectory, illustrated in Figure 2.3.

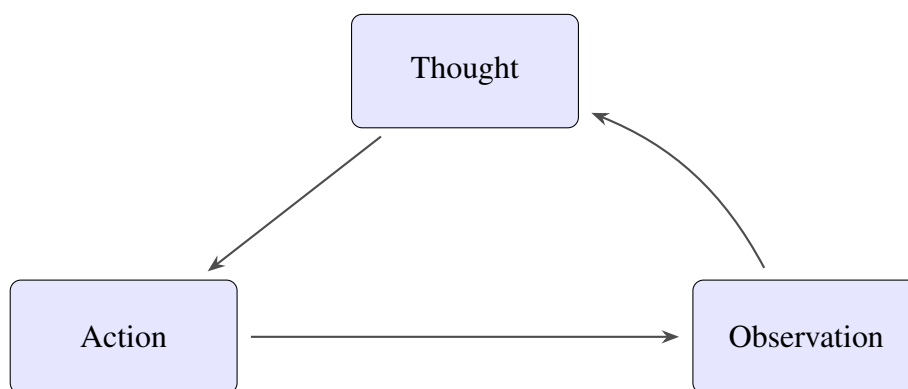


Figure 2.3: The iterative execution cycle of the ReAct framework, combining internal LLM reasoning (*Thought*) and external environment interaction (*Action* and *Observation*), as introduced by Yao et al. ?.

The framework cycles through these three steps until the agent judges the task complete. At the Thought step, the LLM receives the task description, the current environment state, and the accumulated history of prior steps, and generates a reasoning trace about what to do next. At

the Action step, that trace is parsed into a discrete operation; in the web domain this can be a click, a keystroke, or a page navigation. Once the action has been taken, the Observation step captures the resulting page state and appends it to the context for the next Thought step. Because each reasoning step is conditioned on what the agent actually sees rather than on what a designer anticipated, the loop can accommodate mid-task exceptions and changing page states without scripted fallback logic.

The main limitation of this loop is context accumulation. Each observation grows the prompt, and on long-horizon tasks the earliest steps drift into the middle of an increasingly long context, where models use information least reliably.

Since its publication, ReAct has become the standard execution loop for LLM-based web agents. It is the prebuilt default agent in LangGraph, a documented agent pattern in AutoGen, and the architecture of the baseline agents in WebArena and BrowserGym, the evaluation frameworks used in this thesis. Numerous extensions have since been proposed; two are worth distinguishing from ReAct for a complete picture of the most common LLM-based agents. Reflexion adds a self-reflection stage between attempts at a task: after a failed trial, the LLM generates verbal feedback on its own trajectory and stores it in an episodic memory buffer that conditions the next attempt. Tree of Thoughts explores several reasoning branches in parallel rather than committing to a single trajectory, which helps on tasks where early choices are hard to reverse, but produces non-linear trajectories that do not map cleanly onto the memory extraction pipeline this thesis uses.

Discussion

Since the focus of this thesis is the contribution of shared memories for agents, any other aspect of the methodology should be kept as simple as possible. Thus relying on the industry baseline of the simple ReAct agent, rather than a more complex framework, is the more reasonable approach. This choice is further supported by the fact that the framework, despite its simplicity, is still effective. Furthermore, the memory framework used in the paper that provided the baseline for the private memory generation was also implemented on the ReAct framework. By using the same mechanism, this thesis' contribution could more easily be compared to said paper.

2.1.4 Memory architecture

The context accumulation problem identified in Section 2.1.3 points to a more fundamental constraint that has not yet been addressed. The context window is the only memory an LLM natively has. When a task ends, its trajectory is discarded, and the next task begins from a blank slate. This implies that an agent that discovered a successful strategy yesterday must rediscover it today. A memory module addresses this by maintaining a store outside the context window, together with procedures for writing new entries into it and reading relevant entries back into the prompt. Because full trajectories grow quickly, and the longer they grow the less reliably the model attends to them, the store typically holds condensed natural-language records of what mattered in an interaction, rather than its full transcript.

How such a store should be organised is most influentially answered by the CoALA framework, which transfers the memory taxonomy of cognitive science to language agents. CoALA divides memory into a short-term working memory and three long-term stores. The working memory holds the information about the current circumstances. In a web agent, this includes all relevant information from the current trajectory, including task description, the reasoning trace, and the recent observations circulating through the ReAct loop. This information is discarded as soon as the trajectory ends. The episodic memory picks up these discarded trajectories and

records key information from them for later use. When a new task triggers the memory store, only a subset of memories, ideally the most relevant memories for the task, are added to the context window from the episodic memory, which can then help guide the agent. Whereas the episodic memory stores information on the agent’s past trajectories, the semantic memory holds facts about the world or the agent itself. These facts are generated independently from the agent trajectories and are often based on internal documents, which are relevant to the agent. The procedural memory contains the procedures that run the agent, and which must be included in every agent run. The tiers differ not only in content but in how they are written: episodic memory grows with every completed episode, semantic memory through the extraction of facts from alternative knowledge sources, and procedural memory only through deliberate modification, since a faulty write there alters the agent’s behaviour everywhere.

Architectures built on this structure differ mainly in how entries are created, retrieved, and maintained as the store grows. MemGPT [1] frames the problem as memory management, in analogy to an operating system paging data between fast and slow storage. Its prompt is divided into three parts: read-only system instructions, a fixed-size working context holding key facts about the user, and a queue of the most recent messages. An important distinction with this system is that the working context is bounded by design, so it must be rewritten as the conversation moves on if it is to reflect the most important and up-to-date information only. The model performs these rewrites itself, through self-issued function calls that move information between the prompt and external storage, sustaining interactions that exceed the context limit. Generative Agents [2] instead concentrate on retrieval and consolidation. Each agent appends every experience to a memory stream and scores entries at retrieval time by a weighted combination of recency, importance, and relevance, injecting the top-ranked memories that fit within the context window. Once the accumulated importance of recent events crosses a threshold, the agent synthesises groups of related memories into higher-level reflections, which then re-enter the stream as memories in their own right. MemoryBank [3] addresses maintenance, so that a store accumulated over months of interaction does not retain everything indefinitely. Each entry carries a retention score that decays exponentially with the time elapsed since it was last recalled, following the Ebbinghaus forgetting curve, and each retrieval raises the entry’s memory strength. Entries that are never recalled therefore decay out of the store, while repeatedly recalled ones persist.

This design space has carried over into production systems. Mem0 [4] runs a two-phase pipeline in which an LLM first extracts candidate memories from recent conversation and then decides, for each candidate, whether to add it to the store, merge it into an existing entry, delete an entry it contradicts, or leave the store unchanged. LangChain’s memory SDK organises its long-term store into the episodic, semantic, and procedural types of CoALA [5]. The taxonomy has, in effect, become the shared vocabulary in which agent memory systems are specified.

Table 2.1 sets these architectures against one another. The first three columns show that they divide the memory problem between them, each taking on a different operation. The final column shows what they have in common: in every case the store is bound to one user or one agent. What one user’s agent learns is never available to another, and where platforms do permit sharing, no formal bound is placed on the private information that crosses the boundary. This is the gap the thesis addresses.

Discussion

This thesis operates within the CoALA taxonomy, as it is the widely accepted industry and academic standard. However, given the limited scope of the thesis, it only implements the working and episodic memory and ignores the use of the semantic and procedural memory for

Table 2.1: Memory architectures for language agents, by the memory operation each one addresses and the scope of the resulting store.

System	Year	Stored unit	Operation addressed	Scope of store
MemGPT ?	2023	Fixed-size working context; external archival storage	Management: self-issued function calls page data between prompt and storage	Single agent–user conversation
Generative Agents ?	2023	Natural-language observation in a memory stream	Retrieval and consolidation: scoring by recency, importance, relevance; periodic reflection	Single agent
Memory-Bank ?	2024	Conversation record, event summary, user portrait	Maintenance: retention decaying with time since last recall, reinforced on recall	Single user
Mem0 ?	2025	Extracted fact	Consolidation: each candidate resolved against the existing store	Single user

now. It also does not implement any of the more advanced memory architectures described above, as these would complicate the creation, maintenance, and sharing of user memories if they are replaced, erased, changed, or consolidated mid-way. The working memory is realised by the ReAct loop of Section 2.1.3, and the episodic memory provides the top-k relevant tasks. It is this episodic memory which forms the basis for the proposed shared memory layer that is implemented in this thesis.

2.1.5 Memory creation and retrieval

While the previous section set out how a memory system for an LLM may be structured, this section covers how the episodic memories that populate it are created and retrieved.

Memory creation

The question of memory creation does not ask how the store is managed, but what should be written into it once a task ends. This is a growing field of research, and this section covers the most well-known approaches. Reflexion ? is the simplest case. After a trial, the agent writes a verbal self-reflection into a buffer that is bounded to between one and three entries for context reasons, and retries the same task with that text in its prompt. Voyager ? moves from reflections on failure to reusable competence. Operating in Minecraft, it stores each verified behaviour as a program in an ever-growing skill library, so that skills compose into more complex skills and transfer to a new world. ExpeL ? lifts the Reflexion idea beyond a single task instance. During a training phase it gathers trajectories across a set of tasks, then compares success and failure pairs to derive natural-language insights, which it maintains through four operations that add, edit, upvote, or downvote an insight as further evidence arrives. At test time the agent receives both these insights and its most similar past successful trajectories as few-shot examples. AWM ? applies the skill-library idea to the web. From successful trajectories it induces workflows, each one a high-level goal description paired with a routine of actions, with example-specific values

abstracted away so that the routine transfers to other tasks. Workflows can be induced offline from annotated examples or online from test tasks as the agent solves them, and an induced workflow can serve as a building block for a more complex one. ReasoningBank ? distils structured items consisting of a title, a one-sentence description, and the strategy content, and does so from both successful and failed trajectories. Its success labels come from an LLM acting as a judge rather than from ground-truth annotation, which allows the store to grow without supervision. Table 2.2 summarises these frameworks.

Table 2.2: Frameworks for creating episodic memories, by what each one writes to the store and how far that entry transfers.

Framework	Year	Unit written	Written when	Transfer beyond source task
Reflexion ?	2023	Verbal self-reflection on a trial	After each trial	None; reused only on retries of the same task
Voyager ?	2023	Executable skill program	After self-verification confirms the behaviour	Yes; skills compose and carry to a new world
ExpeL ?	2024	Cross-task insight; past successful trajectory	During an offline training phase over a task set	Yes; applied to unseen test tasks
AWM ?	2025	Workflow: goal description and action routine	From successful trajectories, offline or online	Yes; specific values abstracted out, workflows compose
Reasoning-Bank ?	2025	Strategy item: title, description, content	After every task, success or failure, labelled by an LLM judge	Yes; strategies abstract from surface task detail

Memory retrieval

Creating memories is only half the problem. A store that grows with every completed task cannot be placed into the prompt in full, so the agent needs a way to score each entry against the task in front of it and keep only the highest few. This problem is true for both the episodic as well as the semantic memory, which can both be larger than the context window of an agent. The long-standing answer is to match words from the task to the memory store. BM25 ? is the canonical form of this approach. It scores a memory entry on the task terms it contains, weighting rare terms more heavily than common ones, applying diminishing returns so that repeating a term cannot dominate the score, and correcting for length so that long entries do not rank highly on volume alone. The method needs no training and runs over an inverted index, which makes it cheap to build and fast to task. However, BM25’s significant limitation is that it can only find an entry that shares wording with the task, so a memory describing the same strategy in different words is invisible to it.

The alternative is to match meaning. Dense retrieval encodes the task and each stored memory as vectors, positioned so that related texts sit close together, and scores a pair by how close the two vectors are. Dense Passage Retrieval ? (DPR) is the standard implementation. It uses two separate BERT encoders, one for tasks and one for the entries being searched, and scores a pair by the inner product of their vectors, though cosine similarity is the equivalent choice

where vectors are normalised. Encoding the two sides independently matters in practice, because each stored entry can then be embedded once when it is written and reused on every subsequent task, rather than rescored from scratch each time. A mechanism that implements this pattern is called Retrieval-Augmented Generation [2]. It retrieves the top- k entries for a task and places them in the prompt as context alongside it. In the paper that sets out this mechanism, Lewis et al. retrieved passages from Wikipedia to answer questions, which is closer to a semantic memory implementation than episodic. However, for agents, the RAG retrieves memories from the agent’s own past trajectories to guide a task, thus making it an episodic memory implementation.

Neither approach dominates. On the open-domain question answering benchmarks it was built for, DPR retrieved the correct passage within its top 20 results 9 to 19 percentage points more often than a strong BM25 baseline [2], and dense retrieval became the default on that evidence. The picture changes once a retriever is moved off the data it was trained on. Thakur et al. [2] evaluated ten systems across eighteen datasets in the BEIR benchmark and found that BM25 remains a robust zero-shot baseline which dense models can substantially underperform, and that in-domain accuracy is a poor predictor of how a model generalises.

Both BM25 and DPR face three key limitations, which apply to this thesis’ work as well. The first is that similarity does not necessarily equate to usefulness. Retrieval ranks memories by proximity in embedding space, which is only a proxy for whether a memory helps on the task at hand, and a task phrased similarly to the current one may still have been solved by the wrong strategy. The second is that a fixed cutoff returns a fixed number of entries whether or not the store holds anything relevant. This means that the agent is handed weak matches early in a run, which then occupy context space in the prompt, which may misdirect the model’s attention [2]. The third is the domain shift that BEIR documents, which is that the encoder used to embed memories was not trained on agent trajectories, so its notion of similarity is not guaranteed to be the one that matters here.

Discussion

This thesis nonetheless adopts a plain dense retriever over a small fixed number of memories, with the parameters given in Chapter 4. The purpose of the project is not to improve memory retrieval but to measure what cross-user aggregation does to the memories themselves, and holding retrieval fixed and simple isolates that effect, keeps the scope within the limits of an MSc thesis, and leaves any observed change attributable to the memory layer rather than to the retriever. One consequence does need to be carried forward. Retrieval is used at two points in the proposed system, once to select memories for the agent and once when memories from separate users are grouped before aggregation. The second use is constrained by the privacy mechanism of Section 2.1.6, which restricts what a grouping decision may depend on.

2.1.6 Differential Privacy

The sections above establish an agent that improves by writing memories about its own trajectories. So far those memories are either kept private to the user who generated them, in which case nothing is learned across users, or opened up to everyone. The second option is only safe under the assumption that a memory distilled from one user’s trajectory never carries anything the user would not want the rest of the organisation to see. This assumption does not necessarily hold in an agent setting, as described in Chapter 1.

The natural response is to ask a model to strip out anything sensitive before the memory is shared. This is not a robust solution, a fact is sensitive depends on what else a reader already knows, and no filter can enumerate the auxiliary information a reader might hold. The problem

calls for a definition that bounds what can be inferred from a shared memory whatever the reader already knows, and that holds without having to name in advance the facts it is protecting.

The Theorem of Differential Privacy

Differential privacy provides that definition. A randomised mechanism M satisfies (ε, δ) -differential privacy if, for every pair of neighbouring datasets D and D' and every subset O of the mechanism's range,

$$\Pr[M(D) \in O] \leq e^\varepsilon \Pr[M(D') \in O] + \delta, \quad (2.1)$$

where the probability is taken over the mechanism's own randomness(?). The parameter ε bounds how much the distribution over outputs may shift when one dataset is exchanged for a neighbouring dataset, while δ is the probability that this bound fails. A neighbouring dataset can be defined in three different ways:

- Under **add/remove** differential privacy the neighbour is obtained by adding or removing a single record (?).
- Under **replacement** adjacency the neighbour is obtained by replacing one record with another, so the dataset size is held fixed. claude: what is the most relevant reference? See from the invisibleink paper and go from references from there
- Under **replace-by-null** adjacency the neighbouring dataset replaces one record with an empty string. This is strictly speaking a form of replacement adjacency, but with an empty replacement value rather than an actual datapoint.

Since the theoretical proofs underlying any differentially private mechanism differ depending on the adjacency type, it is important to be aware of the neighbouring dataset definition for any methodology. This thesis relies on replace-by-null adjacency. Irrespective of the adjacency type of neighbouring datasets, (ε, δ) -differential privacy always guarantees that for every pair of neighbouring datasets, the absolute privacy loss incurred by an observed output is bounded by ε with probability at least $1 - \delta$ (?, Lemma 3.17). It follows that δ must be kept well below the inverse of the dataset size so that at $\delta \approx 1/n$ a mechanism can satisfy the definition while publishing the complete records of a small number of participants (?). The guarantee is far-reaching, as it is distribution-free.

The second distinction that differential privacy requires users to make, besides the definition of neighbouring datasets, is whether the proof is for example-level or user-level privacy (?, §2.3.3). Under **example-level privacy** a record is a single entry, and the mechanism must hide the contribution of one memory. Conversely, under user-privacy a record is everything one user contributed. Here, the mechanism must hide that user's entire history (?). Example-level protection of a single memory is weaker than protecting a user outright, but it is also easier to enforce.

Differential Privacy for LLMs

Work applying differential privacy to language models falls into three families, separated by where in the pipeline the privacy cost is paid (?):

- **Private fine-tuning** spends it during training. The LLM is trained on a private dataset, but the learning mechanism is adjusted to ensure example-level privacy using a differentially

private stochastic gradient descent (DP-SGD) which perturbs per-example gradients so that no single training example moves the weights too far. (2), and (3) used this to fine-tune an LLM that then emits synthetic text.

- **Private filtering** spends the privacy cost at selection time, where a model with no access to the sensitive data proposes candidate outputs and the sensitive data is used only to score or select among them.
- Lastly, **private prediction** spends the privacy cost at inference time. The model is never trained on the sensitive data at all, and privacy is enforced on each prediction the model makes from that data (4). An example implementation is a sample-and-aggregate mechanism, where sensitive data each produce a non-private prediction and the predictions are then aggregated under noise, as in PATE (5).

Fine-tuning is a poor fit for this project because the shared memory store changes after every completed workflow, and each change would require retraining. Filtering is an equally poor fit because the private memories are precisely the content that has to reach the shared store, and a filtering mechanism never lets that content into the output. This mechanism could only work if the LLM already had a tendency towards proposing text similar to the private information. The private prediction is the mechanism that matches the problem, since it aggregates several private inputs into one output without touching the model weights.

Implementing private prediction

Applied to text, private prediction treats each sampled token as a prediction and combines these predictions into a perturbed new token distribution. Since this method is done for each token, the privacy cost accumulates with each new token generated. This is the binding constraint, and it is why earlier methods in this family produced fewer than ten synthetic examples at usable privacy levels (6). Before the publication of the (7) paper, private prediction was already used to protect data that enters a model at inference time. (8) generate synthetic few-shot demonstrations under a formal guarantee and place those in the prompt instead of the real examples. (9) take a noisy vote across an ensemble of prompts to the same effect. Closer still to the present setting, (10) and (11) apply differentially private token generation to the documents a retrieval-augmented system pulls into the prompt, with (12) spending budget only on the tokens that carry sensitive content. However, (13) was a foundational paper, as it managed to reduce that cost in three ways:

- The first replaces the Gaussian or Laplace noise that earlier methods added to the token distribution with a clipped average of the per-prompt logits, sampled through a softmax. This distorts the distribution less.
- The second concerns how prompts are grouped. Where (14) draw a fresh random subset of prompts for every token, Amin et al. fix one disjoint batch per generated example and pay for it only once, via parallel composition¹.
- Third, they compare the private token distribution against one produced by a model with no access to the sensitive data, sampling from the public distribution whenever the two are close, so that predictable tokens cost nothing².

¹Parallel composition states that if a *roh*-zCDP mechanism is run independently on disjoint batches, then the overall mechanism is still *roh*-zCDP. In the context of the Amin et al. paper, this means that the privacy cost does not accumulate across batches.

²On structured datasets only around 20% of tokens end up drawn privately

While these ease the privacy constraint, the mechanism does impose one important requirement on the system, stated as Assumption 1 of the paper:

A prompt's assignment to a batch must depend on that prompt alone and not on the other prompts in the dataset.

This constrains how memories may be grouped before aggregation, and required a smart solution from this thesis' methodology.

The latest research that fundamentally built on the work that ? published is ?. Their contribution is two-fold:

- An improved **clipping** method, where only the sensitive information is clipped rather than the entire logit vector.
- **Top_k**+decoding, where the softmax only samples from the top k tokens, rather than from the entire ranking tokens from entering the space, even when clipping the logit vectors.

Their contribution drastically decreased the required privacy budget needed to generate a sensible sentence such that a batch as small as $s=3$ produced feasible results. The paper states that they can "produce hundreds of tokens at 80% lower computational cost at the same privacy/utility trade-offs compared to prior SoTA methods"³. The mechanism is described in further detail in section 4.2, as it is the mechanism this thesis builds on.

Discussion

What the work above shares is a static private dataset, a single aim, and a distinct boundary between private user and viewers of the public data: The sensitive data is a fixed collection of documents or labelled examples, which does not grow or change over time. It is consumed to answer one task or generate a single dataset, and there is a clear distinction between the creators of the corpus and whoever reads the output. The setting of this thesis differs on all three counts. The corpus is written by the agent about its own trajectories and grows continuously as tasks complete, so the privacy budget is spent repeatedly over the lifetime of the deployment rather than once. The consumers of a shared memory are the other users of the same system, so the boundary runs between users rather than around the corpus. And the artefacts being protected are distilled reasoning strategies rather than source documents, which means the aggregation step has to preserve transferable procedure while discarding the user-specific detail it was learned from. Existing systems that do share memory across users address the boundary with access control, as in the provenance and permission model of ?, or with redaction, but neither yields a quantified bound on disclosure. It also leaves room for error on sensitive information that was not clearly scoped within the redaction rules. Applying differential privacy to agentic memories shared between users is, to the best of the author's knowledge, unaddressed, and it is the gap this thesis occupies.

2.2 Research question

Each component of this problem has been solved on its own. Memory frameworks raise agent performance on repeated tasks, and every framework covered in Section 2.1.4 keeps a user's memories for that user. Differential privacy bounds disclosure without assuming anything about what an adversary already knows, and private prediction carries that bound to text placed in

³The author assumes SoTA to stand for State of The Art in this context.

a model’s context. What remains open is the combination, in a continued deployment setting where the users who supply the memories are also the users who read them.

Given a privacy budget (ϵ, δ) , how much of the performance benefit of unrestricted memory sharing can a tiered memory system recover when its shared tier is generated under differential privacy?

Performance here means task success rate on WebArena, where each task is scored as a pass or a fail. Two conditions bracket the range. An agent with no memory at all marks the floor, and an agent with unrestricted access to every user’s memories marks the ceiling. Recovery is the share of the distance between them that the privacy-constrained system regains.

The naive alternative is to prompt a model to strip sensitive content before a memory is shared. However, this thesis does not treat that approach as a comparator, as it is unable to bound the privacy budget at which it operates. There is no privacy guarantee. The open question is therefore what this theoretic guarantee costs. It is on this basis, that this paper aims to contribute: The shared tier is built by aggregating episodic memories drawn from distinct users, and the aggregation is performed by a private prediction mechanism rather than by a redaction step or an access-control policy. To the best of the author’s knowledge, differential privacy has not previously been applied to agentic memories shared between the users of one deployment.

2.2.1 Hypotheses

The question breaks into two claims, each testable against the scenarios defined in Section 5.2.

- H1.** A shared memory tier distilled from other users’ trajectories raises task success rate above a no-memory baseline. The claim fails if the cross-user condition does not statistically improve on the no-memory condition.
- H2.** That gain survives the privacy mechanism. At $\epsilon \leq 5$, success rate under the differentially private shared tier sits closer to the unrestricted-sharing condition than to the no-memory condition. The claim fails if the private tier recovers less than half the distance between them.

Both hypotheses are precise, testable, and falsifiable, and thus fitting for this thesis’ hypothesis-driven research.

2.2.2 Scope

Given the scope of this thesis, three boundaries had to be set in advance in order to ensure a timely completion. Each narrows what the thesis claims:

- This paper assumes that each of the 812 tasks, on which the agent is evaluated, is run by a separate user. This means that any shared memory draws at most one set of memories from any given user. This is what allows this thesis to infer user-level privacy from the example-level guarantee, since bounding the influence of a single memory bounds the influence of a single user when that user contributes only once.
- The generalisation pipeline is built for episodic memory alone. The architecture in Section 2.1.4 also defines semantic and procedural tiers, and nothing in the mechanism rules them out, but each tier would need its own distillation format and its own judgement about what generalises. In light of the scope of this thesis, the memory architecture is limited to the episodic memory.

- Evaluation runs on WebArena alone. This buys comparability against the memory framework the system extends, which reports on the same benchmark, and an environment that resets to a known state between runs. It buys no evidence that the result transfers to other agentic domains, and Section 7.1 treats that as a limit on the conclusion rather than as a question left open.

Project management

3.1 Agile

Agile is a popular methodology within software development and wider business contexts (?). Whereas other methodologies set out a pre-defined timetable and structure with deadlines and aims attached to each step, the agile methodology is more flexible and accounts for the fact that most software, and in this case research, projects cannot be defined with complete accuracy at the start of the project. The founding principles of agile treat this as a design goal rather than a concession, and ask that changing requirements be welcomed even late in development (?).

The agile methodology was followed in the following steps:

- **Defining the goals.** The aim of this agile method was defined as a SMART goal (?). The aim was to create a complete, self-standing thesis document describing a new methodology of creating differentially private shared memories by the 25th of August 2026. This aim was specific, measurable, assignable, realistic and time-related. Having this aim clearly in mind for each weekly sprint plan allowed the author to keep a clear focus and prioritise where needed. For example, when deciding between code expansion and writing up the thesis, the latter choice was made in later weeks of the project.
- **Product backlog.** The product backlog was initially created from the work packages set out in the proposal. While these were, in particular at the beginning, helpful in defining a structure, later weeks diverged from the exact set of tasks to better suit the requirements of the project. The backlog is defined as an emergent list rather than a fixed one (?), so this divergence is the intended behaviour of the framework rather than a departure from it.
- **Sprints.** Each week marked the beginning of a new sprint, where the author reviewed the items on the Kanban board and decided which items to carry forward, which new items to add, and which were finished.
- **User stories.** The tasks for this project were all grouped under a user story. The stories for this project are listed here:

– “As a Researcher, I want to assemble all proposal sections into a polished, well-formatted document.”

- “As a Researcher, I want to define concrete evaluation metrics and experimental protocols.”
- “As a Researcher, I want to refine my technical architecture and verify tool feasibility.”
- “As a Researcher, I want to create a realistic project plan with work breakdown structure, timeline, and risk mitigation strategies.”
- “As a Researcher, I want to define a precise, scoped research question grounded in essential literature.”
- “As a Researcher, I want to understand differential privacy.”
- “As a Developer, I want to have a clean repo with clear structure.”
- “As a Developer, I want to create a complete architecture in which to run my agent.”
- “As a Researcher, I want to specify a detailed methodology with a formally defined privacy mechanism.”

These stories were all completed with this thesis, and provided a guiding direction for the weekly sprint planning.

- **Refinement.** This was done at the end of every week before planning the sprint for the next week, where the author decided whether the timelines for any item had to be extended or changed, and whether they were still relevant.
- **Feedback.** Feedback was provided by the author’s supervisor, Luca Citi, with whom the author met at a bi-weekly, and later in the process weekly, pace. In addition to receiving structured feedback in each session, these meetings allowed the author to align on priorities, troubleshoot, and discuss risks and impediments. The feedback is recorded alongside the sprint items on the Kanban board.

Two elements of the framework were not adhered to. The first is the daily scrum. Since the author worked by herself, a daily status meeting had no audience and was dropped. The second is the retrospective, which was not followed as a written ceremony but as an internal process on a continuous basis, where the author assessed what worked in her working process and what did not. Since she is the sole contributor to the paper, she did not need to write these out. This thesis therefore implements parts of the agile methodology rather than the framework in full.

Risk was handled at the weekly refinement rather than through a separate risk register. Each week the author reviewed which items were at risk of slipping and decided whether to extend, re-scope or drop them, and the supervisor meetings surfaced the risks that were not visible from the board alone. The pivots that followed, including the move from a cloud instance to local hardware and the change of hypotheses, are set out in Chapter 6.

The agile methodology has allowed the author to re-adjust her targets based on what worked and what did not work. The entire Chapter 6 speaks of the numerous pivots that were required to get this thesis to its current state. The author is therefore content with the choice of methodology.

3.2 Tool use

3.2.1 Tuleap

Tuleap is the university-provided agile methodology tracking tool. All weekly feedback, user stories, and tasks were loaded here. The Kanban board it provides allowed the author to organise

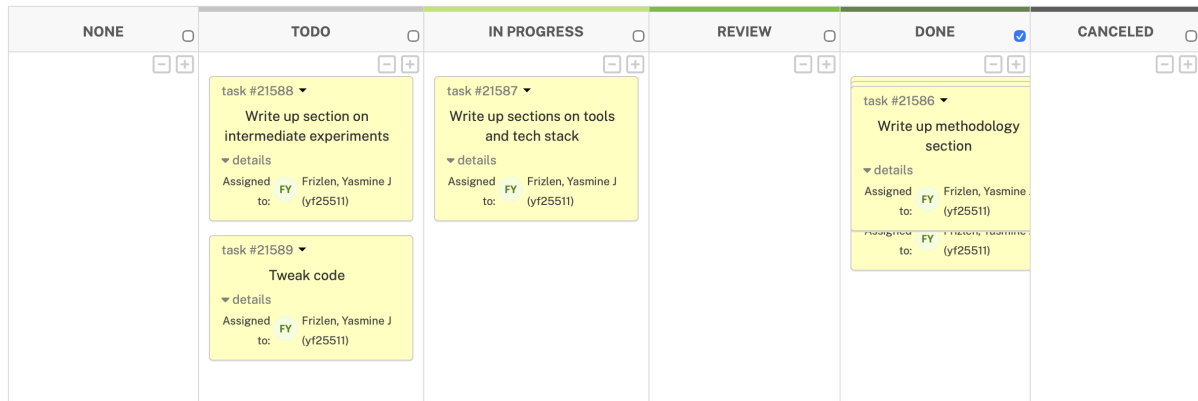


Figure 3.1: Weekly Kanban board excerpt from Tuleap

the weekly sprint planning cleanly, and to move items across the board. A weekly screenshot of a board is included below.

3.2.2 Overleaf

Overleaf was used as the interface for writing this thesis in $\text{\LaTeX}$. The website simplifies compiling the code, and by linking it to GitHub, the source can still be copied to a local repository for offline work. The interface also allows concurrent views of the table of contents, the compiled PDF, and the source, which the author preferred over working in the source alone.

3.2.3 GitLab and GitHub

This author used both GitLab and GitHub as repositories. The GitLab repository, hosted on the university’s `cseegit` instance, is the marked repository and is where all changes, merges, and edits took place. The GitHub repository is a public mirror, kept so that the final repository could be added to the author’s public profile. The two are maintained with the `origin` remote configured with one fetch URL (from GitLab) and two push URLs, so a single `git push origin` updates both GitLab and GitHub at once.

The author implemented a clear structure of creating a new branch for each task, and followed a consistent naming convention of `<task-number>_<name>`, for example `21589_tweak_code`. After adding and committing any local code, it was pushed to GitLab. There, the author would conduct a final review and merge the branch into `master` using the pre-structured merge request template shown in Figure 3.2. The template is again part of the author’s general tech stack and was created as part of the group project module at the University of Essex. It sets out a summary field, a checklist covering tests, code quality, dependencies and documentation, and a block into which the output of the passing test suite is pasted, so that every merge carries its own evidence.

3.2.4 Code quality tooling

This thesis uses code quality tooling to ensure consistent and high-quality code. The checks below are run together with a single `make ci` command, which runs Black in check mode, Ruff, mypy and pytest in sequence and stops at the first failure. The same command is named in the merge request checklist, so no branch reaches `master` without it having passed.

MS

Default.md 680 B

Code

Preview

Blame

Summary

Closes #

Checklist

Testing

- ☐ `make test` passes with no failures
- ☐ New or changed behaviour has a corresponding test

Code quality

- ☐ `make ci` passes with no errors
- ☐ No debug statements or commented-out code left in

Dependencies

- ☐ If new packages were added, `pyproject.toml` has been updated

Documentation

- ☐ New modules or scripts have a docstring explaining their purpose
- ☐ `docs/CODING_STANDARDS.md` is still accurate after this change

Testing evidence

paste here

Figure 3.2: GitLab merge request template

- **Ruff.** Ruff is the linter. It is configured to check pycodestyle errors and warnings, pyflakes rules, import ordering, PEP 8 naming, outdated syntax, and the bugbear rule set, across `src/`, `evaluation/`, `scripts/` and `tests/`. Line-length errors are ignored, since Black already enforces the 100-character limit.
- **Black.** Black is the formatter. It rewrites the same four directories to a fixed style at a line length of 100, which removes formatting from the set of things a review has to consider.
- **mypy.** mypy is the static type checker. It runs over `src/agent_memories/` and `evaluation/` and rejects functions with missing or incomplete annotations, as well as returns that silently degrade to `Any`. Third-party libraries that ship no type stubs, such as LangGraph, BrowserGym and Playwright, are exempted from the import check so that the strictness applies to first-party code alone.
- **pytest.** pytest runs the test suite, which holds 97 test functions across 13 files, split into 12 unit files and one integration test. Tests are marked as `unit`, `integration` or `slow`, and unregistered markers are rejected so that a typo in a marker fails the run rather than silently skipping a test. Line and branch coverage over `src/agent_memories` and `evaluation` is reported on every run, but no minimum threshold is enforced.
- **pyproject.toml.** Rather than use a `requirements.txt` file, this thesis uses a `pyproject.toml` file. The first benefit is that the author was able to differentiate between the packages needed to run the project itself and the packages needed only to develop it, which are declared as an optional `dev` group holding `pytest`, `Black`, `Ruff` and `mypy`. The second is that the same file holds the configuration for all four of those tools, so a rule change is made in one place rather than in four separate configuration files. The third is that the project is installed in editable mode from it, which places `src/agent_memories` on the import path and lets the tests import the package directly rather than manipulating `sys.path`.
- **Helper functions.** The codebase included a Makefile for eight simple commands. These were based on previous work the author had conducted as part of the group project module at the University of Essex, and have become part of her every work cycle since. These included:
 - `make help` Prints a reminder of what each make command does, generated from the Makefile's own comments so that it cannot fall out of date.
 - `make venv` Creates the project virtual environment at `venv/` in the repository root, if it does not exist yet.
 - `make install` Installs the project and its development dependencies, together with the Chromium binary that Playwright drives and the `punkt_tab` tokeniser data, both of which are needed for the WebArena and BrowserGym run.
 - `make format` Auto-runs Black to format the source code in the `src/`, `evaluation/`, `scripts/` and `tests/` directories.
 - `make lint` Finds and fixes the naming, style and imports of the source code with Ruff across the same four directories.
 - `make typecheck` Runs mypy over `src/agent_memories/` and `evaluation/`.
 - `make test` Runs the full test suite with a coverage report that names the uncovered lines.

- `make ci` Runs all four checks in their non-fixing form, so that the command reports what is wrong rather than quietly repairing it.
- **Coding standards document.** The coding standards are the common coding standards this author implements in all her projects, and includes as a generic document to reference. It originated from the same group project module the author had previously partaken in at the University of Essex. It was adapted to this project, and has been added to the annex. The generic sections cover branch naming, commit messages, commit squashing, package structure and dependency management, while the project-specific sections cover the remote configuration and the local model setup that `make install` does not handle.
- **Pre-push hooks.** A single pre-push hook fires the two LLM calls for the `decision-curator` and `thesis-tasks-curator` commands described in Section 3.2.5. The hook runs them in the background and always exits successfully, so a slow or failing curator never blocks a push, and it exits silently if the `claude` binary is not installed. The restriction to the primary branch is not enforced by the hook itself but inside the `thesis-tasks-curator` command, which exits on any branch other than `master`. Because the hook lives in a tracked `.githubhooks/` directory rather than in `.git/hooks/`, it is version-controlled alongside the code and is activated on a fresh clone by pointing Git at that directory.

3.2.5 LLMs

LLMs were used in this thesis as tools to support the author’s progress. While LLMs can be used to do more comprehensive work, this often results in high-level, uncreative, and possibly false results. The author therefore used her subscription to Anthropic’s Claude as a tool, without giving up the ultimate control of the project. The support was divided into the following streams:

- **Logging.** The author created multiple skills to track the work that was being done between each commit, and to log it for future purposes. The author made a clear difference between skills and commands. With Claude, skills can be triggered by anything written in a chat, whereas commands need to be invoked specifically, otherwise they will not run. The prompts for the different skills and commands used are all attached to the annex.
 - First, when pushing to the origin, a `decision-curator` command assesses each decision that was made within the last commit messages, and adds in reasoning, where available in the code, to a growing memory file. This ensured the author automatically grew a log of every decision and change made throughout the code. This decision curator was triggered with a pre-push git hook, so the author never had to remember it.
 - Second, when pushing to origin, a `thesis-tasks-curator` command assessed which thesis tasks, as listed in the original work breakdown structure in the proposal document, had been completed, and how. This allowed the author to have a clean overview of what had been done and what still needed to be done. This command only ran on the `master` branch, so that it would only keep track of tasks once they were truly complete and merged.
 - Third, the author added a `thesis-tasks` skill, which she could invoke to check, update, and add to the outstanding tasks list, which was originally based on the work breakdown structure.

- **Refining text.** The author used a `writing-editing-text` skill to edit her drafts to sound cleaner and more concise. Importantly, this was only used to make the text sound cleaner, and was always based on an existing draft of her own, not to write new sections. The main idea is that she was still the author, with a smart editor giving feedback and checking her writing.
- **Code checks.** On coding, the author used Claude Code to check for bugs in her code prior to pushing to origin. As with writing, Claude was used to ensure the code stayed consistent, in line with the coding standards outlined in `CODING_STANDARDS.md` (see annex), and bug-free. Claude Code was also used as a discussion partner on why certain things may not have been working, and on how to best move forward, on days without a supervisor meeting. It also helped provide a check that all of the test cases were comprehensive.
- **Explanations in code.** I used LLMs to generate summaries of what the code does at the top of each file. This makes the code more accessible to future users by summarising the work I had done.
- **Source finding.** Lastly, the author used Claude Code to check her academic sources, and ensure she did not forget any key sources for a topic. She would then use that information to find the referenced source, and check whether it should indeed be added to this thesis, and if so, in what manner.

Methodology

This chapter describes the methodology of the thesis. The first section describes the system architecture, which should be followed to emulate this thesis' research, and which should be maintained in future implementations. The second section describes the private prediction mechanism, as it is a crucial part of the methodology and required to understand the complete functioning of the system.

4.1 System architecture

The aim of this system is that organisations with private information across their employees can benefit from the learnings of private agents by generating shared memories from the private ones. This system is composed of three different parts, each of which gets triggered at different moments:

- The first system part is the **agent**, which navigates the web. It is the part of the system that the user directs to fulfil their request. A task may include “*Find the cheapest hairdryer on Amazon and purchase it.*” The agent will attempt to solve the task using the observe-think-act reasoning loop from the ReAct framework, and save the final trajectory in the system.
- The second system part is the **private memory creation**, which mirrors the ReasoningBank memory creation. This step can be triggered after a completed trajectory. The benefit of directly triggering the memory creation is that the new private memory can immediately be used for new agent runs.
- The third system part is the **shared memory creation**, this thesis' main contribution. The shared memory generation is triggered in cycles, ideally after a sufficient number of private memories has been generated in order to create useful shared ones. The trigger for a new cycle could be dependent on the number of private memories across different users, a number of calendar days, or another form of regularisation¹.

¹The choice of frequency is only linked to the quality of shared memories in the amount of memories that will have been generated during that period. The more memories are generated, either the more shared memories can be created, or the lower (ε, δ) can be set for the same number of shared memories.

Both the private and shared memory generation contribute to a common embedded storage system of memories, which new agents can select from for guidance. An agent will always have access to the full storage of shared memories, as well as its own private memories.

This system architecture is visualised in Figure 4.1. Each system part's architecture is described in more detail below.

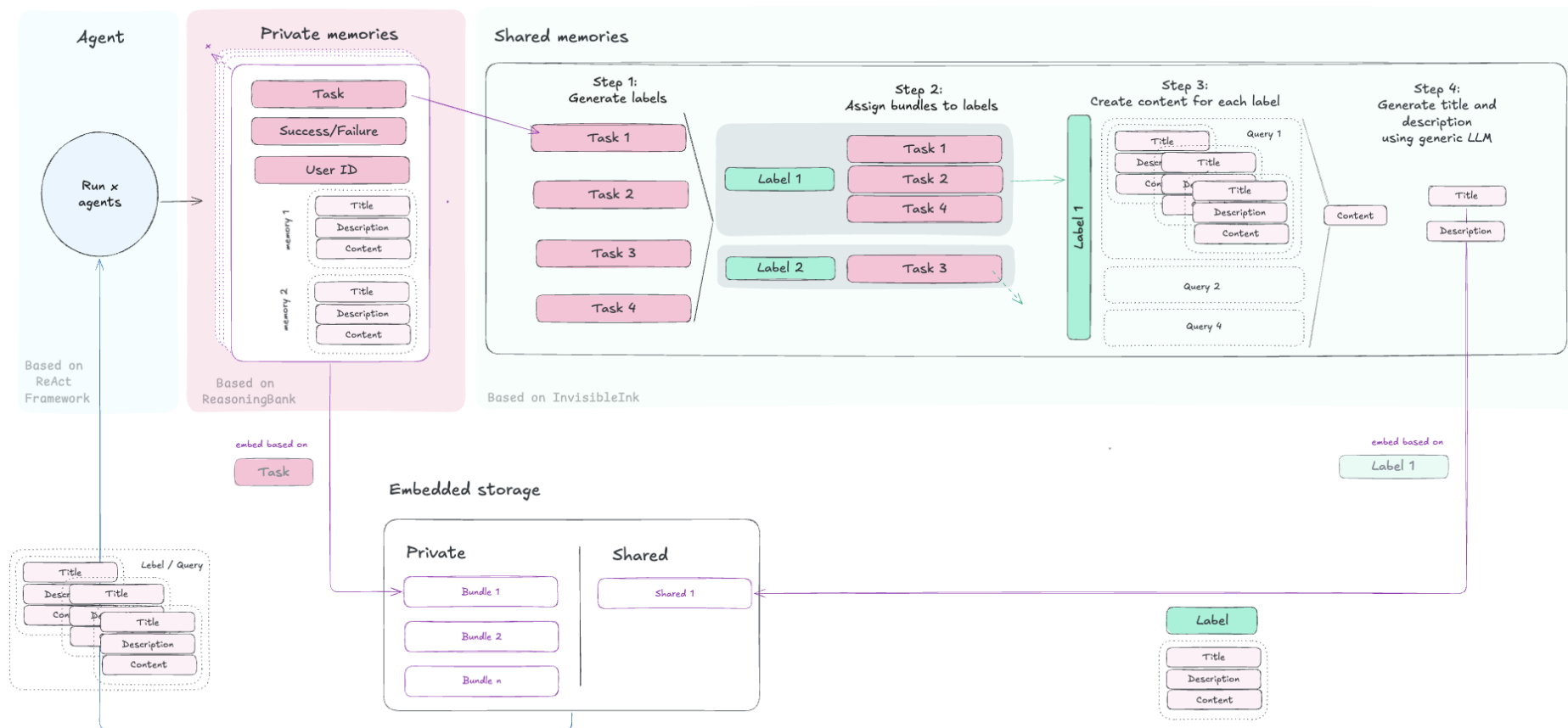


Figure 4.1: System architecture

4.1.1 Agent

The agent is the web-based LLM-powered structure, which attempts to solve a user's task by taking action on the web. The agent implementation is very flexible. There are only two requirements it must fulfill, which are that the agent can provide a complete trajectory to the memory creation mechanism once complete, and that it has the ability to add relevant memories to its context window for improved capability.

4.1.2 Private memory creation

After the agent completes its task, the system triggers the private memory creation with the aim of generating useful learnings from the trajectory. The private memory creation was set up to fully mirror the ReasoningBank approach, as shown in more detail in Figure 4.2. The ReasoningBank paper creates high-level abstractions of what works well, or what does not work well from a given trajectory. In order to do so, it first determines whether a trajectory was successful in solving the initial task² by calling an LLM as judge. On the basis of this decision, another LLM, with a separate prompt, is then called to create the memories.

For each trajectory, up to three memory items are created and held in a single record. Each item includes a title, a description, and the content of the actual memory. Where the extraction returns no parseable item at all, the trajectory is dropped rather than written as an empty record. The record is then stored alongside the user ID, the outcome, and the initial task³. The reason for adding the user ID is so that future agent runs by this user can access their own private memories, as well as so that the sharing mechanism does not accumulate multiple memories of a single user in a batch. The raw trajectory is not kept in the record, since it carries no value for retrieval or for the shared memory creation, and holding it would mean storing long, continuously accruing, trajectory descriptions.

Lastly, this bundle is then embedded as a vector for similarity search for future use cases. The embedding is conducted on the basis of the task. This is because the generated memories contain advice that may divert from what was originally asked in the task, and may therefore not have close semantic similarity to the actual task, even if it is relevant. The original task, on the other hand, is more likely to indicate whether the saved memories are relevant to a future task, if the two tasks are similar in purpose or wording. Thus, each task is stored in the embedded storage for future use.

4.1.3 Shared memory creation

The shared memory creation is the main contribution of this thesis. It is a mechanism which allows continually accruing private memories to be transformed into shared memories, which can then be used to guide future iterations of agents. The shared memory creation is formed of four steps, as shown in the detailed breakdown in Figure 4.3. Step 1 and 2 create buckets for the new private memories, while step 3 and 4 create the content, description, and title for the new shared memory per bucket⁴.

²The ReasoningBank paper refers to tasks as queries.

³This is a departure from the original ReasoningBank approach, where the user ID is not stored since no differentiation between user memories was made.

⁴Steps 1 and 2 are not strictly required, as all memories could in theory be put into one bucket and new memories derived from there. However, grouping together similar memories for the creation of a new shared memory seemed intuitively more sensible, as it would improve the specificity of the new memory being created.

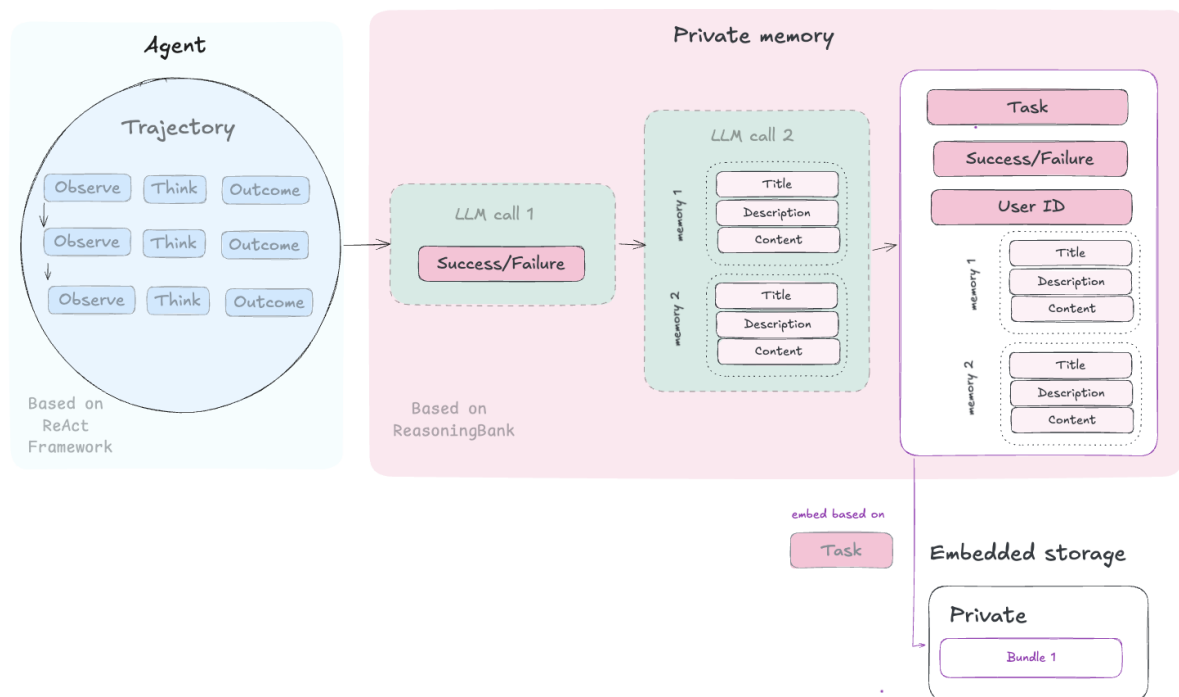


Figure 4.2: Private memory mechanism

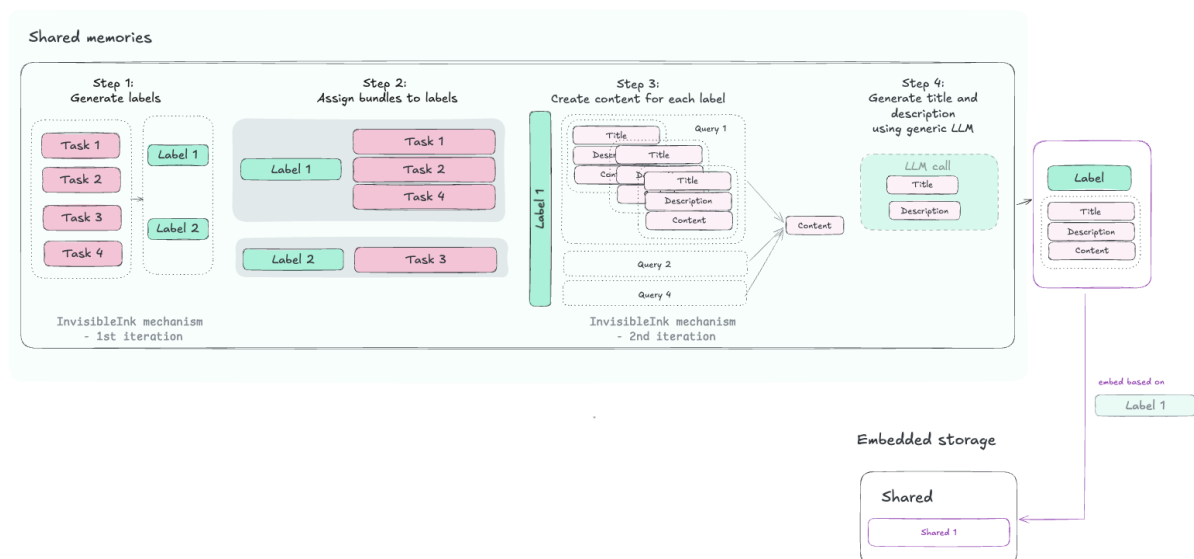


Figure 4.3: Shared memory generation architecture

Step 1 - Label generation

The label generation uses the differentially private algorithm, as described in more detail in section 4.2 to generate a pre-defined number of labels across the tasks.

The reason for using the differentially private algorithm, rather than a simple hash function, as in the original ϵ algorithm, or a ordered list, as in the δ , is that the usefulness of a shared memory depends on how far the private memories underlying it agree with one another. Each differentially private token is sampled from a clipped and averaged batch of token distributions, so a token only carries weight if several batch members support it. Where the private memories are completely different, no content token gets support from more than one member. As a result, the averaged distribution flattens, and the sampled token tends towards whatever is generically likely in English rather than towards the lesson the memories hold in common. This severity of this issue has been reduced with the Top_k+ sampling method of the ϵ paper over the original ϵ paper, but is still consequential. Grouping semantically similar memories raises the agreement between batch members, and with it the chance that the tokens carrying a shared lesson survive the averaging. A partition that ignores what the memories say cannot produce that agreement.

The first choice was therefore to group similar memories together. This led to the question of how to do so. Amin et al.'s Assumption 1, on which ϵ relies as well, requires each task's batch assignment to depend only on that task itself, and not on the other tasks in the dataset. This means that adding or removing one task from the dataset does not change which batch other tasks land in. Thus, any traditional clustering approach such as k-means clustering is not valid under the differentially private algorithm⁵. Furthermore, such a clustering approach uses the private information to cluster, and thus is no longer differentially private.

A different approach was therefore required, which both ensured independent batch assignment, and differential privacy. The simple solution to this problem was to take a two-step approach where the ϵ mechanism was used to generate the labels and memories would then be assigned to these labels independently in Step 2 of this thesis' mechanism.

The additional benefit of using the InvisibleInk algorithm was that no further analysis was needed to deduce the privacy guarantee of the entire mechanism. Each memory passes through the algorithm twice, once in Step 1 and once in Step 3, so the two costs compose sequentially and the total privacy budget is simply $(2\epsilon, 2\delta)$.

Thus, the tasks are used to generate a set number of new labels using the InvisibleInk approach. These are then parsed out of the differentially private output.

Step 2 - Bundle assignment

The second step in the shared memory creation is assigning the tasks to labels. The mechanism does not prescribe how many tasks to a label, as this would contradict Assumption 1 of the algorithm. What makes the second step legitimate is that the labels are themselves created under differential privacy. Any output of a differentially private mechanism is public with respect to what follows it(?). Each task is therefore assigned on the basis of its own embedding and of public labels alone, which is what Assumption 1 asks for. Assigning tasks to a label is done by embedding the labels, and assigning each task to the label with the highest cosine similarity. While the algorithm does not prescribe the number of elements to a label, it does specify how many tasks must have been assigned to a label, for that label to move to Step 3 of the shared generation mechanism:

⁵Since adding or removing one task from the dataset could change the cluster centers, and thus the cluster assignments.

- If x or more memories are grouped under one label, then this bundle of memories moves to Step 3 of the shared memory creation.
- If less than x memories are assigned to the label, then no shared memory is created from them. This design choice was made because the privacy budget (ϵ, δ) is linked to the number of memories used for the algorithm. In order to maintain the same privacy budget, but with less memories, the obfuscation parameters have to be increased substantially to maintain the balance. Instead of generating a subpar shared memory with the insufficient number of private memories, these memories are instead put on hold for future cycles of the shared memory creation. In these future cycles, these memories are exempt from step 1⁶, but are reintegrated into the loop as of step 2, by being assigned to the newly created labels based on their embedding value.

Gating a label on the number of memories assigned to it is itself dependent on the content of those memories, since whether a label reaches x depends on what the users happened to ask that cycle. The gate therefore leaks information about how the private memories are distributed across topics, which the per-step privacy budget does not account for. The design accepts that leak in exchange for the quality gain of a full bucket. The alternative is for shared memories to be created irrespective of the number of tasks that are grouped under the label. This quality-privacy trade-off is a choice the implementer needs to make.

Step 3 - Content generation

The labels with sufficient memories are then moved to Stage 3. Stage 3 runs the differentially private mechanism once more, this time to generate the shared memory from the examples. The algorithm is run for each bucket of memories attached to a label that passes into this step, and produces one shared memory per bucket.

The mechanism generates the content field of the new memory only, and not its title or description. Markdown scaffolding such as a `## Title` header would simply consume unnecessary privacy budget that would otherwise go to the lesson itself. Generating prose only, and deriving the remaining fields afterwards, removes this problems.

Step 4 - Title and description generation

This step could theoretically be skipped, but in order to keep shared memories as similar to the private memories, and to keep as close to the ReasoningBank framework as possible, we generate the title and description for the new shared memory as well. This last step is completed with a generic LLM, which is prompted to provide a title and description for the new memory content. The call is not differentially private and does not need to be. Its only input is the content released by Step 3, which is already a differentially private output, and post-processing a differentially private release costs no further budget (?). Together with the label and the content, the title and the description are then grouped together in one entity. As with the private memories, this entity is transformed into an embedded vector. This time, the embedding is not done based on the task, since multiple tasks and memories led to the shared memory, but based on the label which grouped them.

⁶They are exempt because their privacy budget has already been used once for Step 1

4.2 Differential privacy mechanism

The differentially private mechanism applied in this thesis' final implementation is the InvisibleInk algorithm[?], which builds on the [?] paper. The algorithm is applied without any changes, and departs from the paper only in how it is deployed, as set out at the end of this section. It is used twice throughout this paper's methodology, once to create the labels (Step 1 of the shared memory creation) and once to generate the content of the shared memories (Step 3). Each time the algorithm is applied, it goes through each of the IV sections below. The author's division of the algorithm into the IV sections is solely done for ease of understanding and has no mechanistic purpose. While Algorithm 1 (an exact copy of [?]) sets out the mathematical implementation, Figure 4.4 provides the simplified, and hopefully more intuitive illustration.

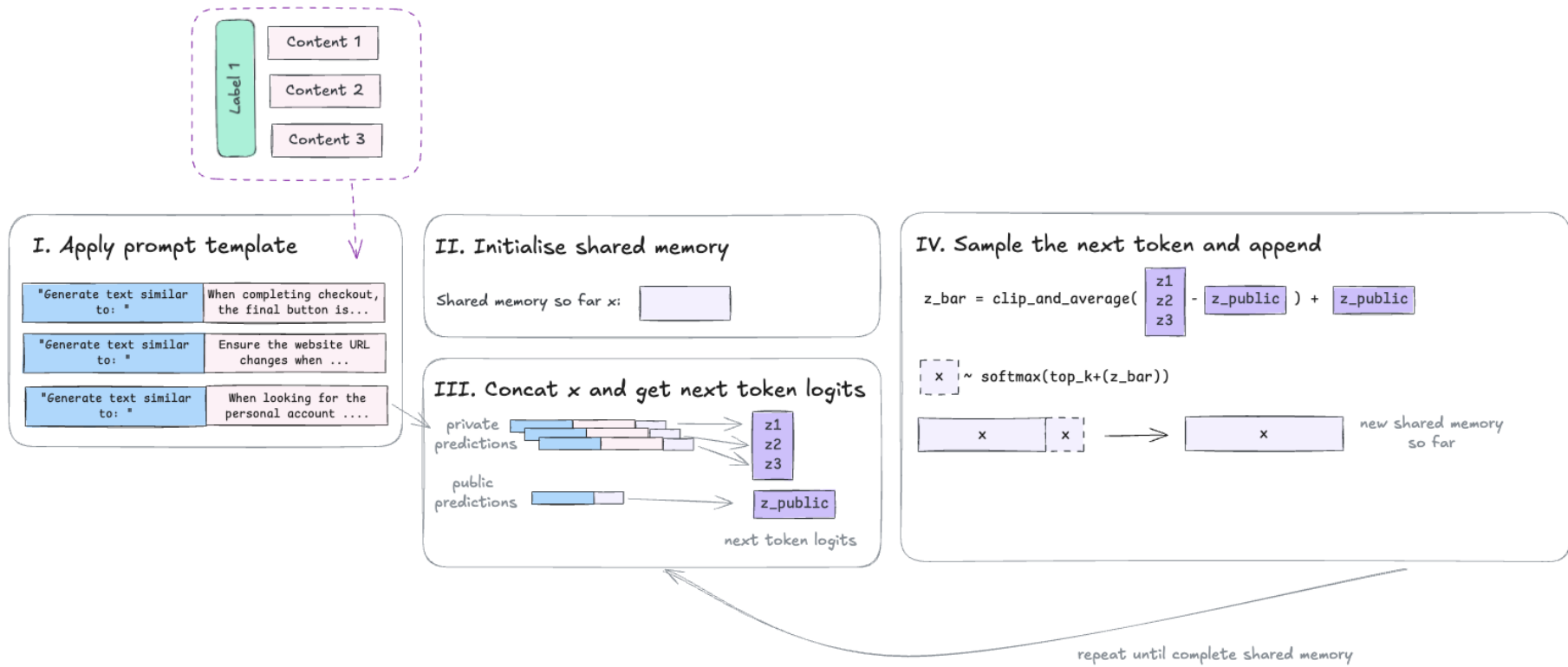


Figure 4.4: Differential privacy mechanism

Algorithm 1 INVISIBLEINK for DP Text Generation

Require: LLM logit API $\phi(\cdot \mid \cdot)$, vocabulary V , query q , sensitive references $R = \{r_1, \dots, r_B\}$, max text length T , clip norm C , temperature τ , top- k parameter, initial generation $x = \emptyset$

- 1: **for** $t = 1, \dots, T$ **do**
 - 2: For $i \in [B]$, set $\phi_i = \phi(\cdot \mid q, r_i, x_{<t}) \in \mathbb{R}^{|V|}$ and $\phi_{\text{pub}} = \phi(\cdot \mid q, x_{<t}) \in \mathbb{R}^{|V|}$ $\triangleright$ Section III: LLM calls
 - 3: $V_k^+ = \text{EXPANDEDTOPVOCABULARY}(\phi_{\text{pub}}, k, C, B)$ $\triangleright$ Vocabulary for Top- k + sampling
 - 4: Set $\bar{\phi} = \phi_{\text{pub}} + \frac{1}{B} \sum_{i=1}^B \text{clip}_C(\phi_i, \phi_{\text{pub}})$ $\triangleright$ Section IV: Aggregated clipped logits
 - 5: Sample $x_t \sim \text{softmax}(\bar{\phi}[V_k^+]/\tau)$ $\triangleright$ Exponential mechanism over V_k^+
 - 6: **Yield** next token x_t , append it to x , and **break** if $x_t = \langle \text{eos} \rangle$ $\triangleright$ Generated Token
 - 7: **procedure** $\text{EXPANDEDTOPVOCABULARY}(\phi_{\text{pub}}, k, C, B)$
 - 8: Set ℓ to be the k^{th} largest entry of ϕ_{pub} $\triangleright$ top- k threshold of ϕ_{pub}
 - 9: **return** $\{y \in V : \phi_{\text{pub}}(y) \geq \ell - 2C/B\}$ $\triangleright$ Expand top- k threshold of ϕ_{pub} by $2C/B$
-

The four sections are described as implemented for Step 3 of the shared memory generation, i.e. the content generation.

Section I - Apply prompt template

Each task that has been assigned to the same label is attached to the prompt template⁷. The instructions ask the LLM to generate a similar text to the memory it is being given. Each memory becomes one prompt, and the bucket becomes one batch. The template is identical across the batch, so the only thing that differs between two prompts is the private memory each one carries. In addition to each private memory being attached to the prompt, the algorithm also keeps a separate prompt without attaching any memories. This forms the basis of the public LLM response - which is not private and can therefore be used without a reduction of the privacy budget.

Section II - Initialise shared memory

When the algorithm is first called, no shared memory exists yet. It is thus initialised as an empty string. With each cycle a new token is added to this shared memory in line with sections III and IV until a complete shared memory has been created.

Section III - Concat x and get next token logit

The shared memory block is appended to each entity from section I. For each entity, this three-part prompt is then fed into an LLM to generate the next token logit vectors. The logit vectors for each memory (z_1, z_2, z_3) are each the size of the dictionary, and represent how likely each of the tokens in the dictionary are to follow the injected prompt.

The algorithm then determines what tokens in the dictionary fall under the ExpandedTopVocabulary (Top- k +). This vocabulary is a subset of the most likely tokens according to the token distribution of the public logit vector. The threshold is set by both the choice of top k tokens, as well as the privacy parameters C (the clipping bound), and B (the batch size).

⁷Each task may contain up to three memories. These are all included as part of one LLM prompt. While they could have been separated to triple the batch size, the author decided on implementing the simplest approach given the short duration of this project. Further work could be conducted to compare the quality of shared memories when combining the three private memories or when separating them out.

Section IV - Sample the next token and append

The algorithm then aggregates and clips the logit vectors. It does so by clipping the difference between the logit vectors of the private and public distribution and adding this clipped average to the public logit vector. The reason for clipping is to ensure no single private memory can shift the output of the combined distribution too much, as this would break the underlying differential privacy guarantee. This algorithm cleverly chooses to clip the difference between the private and public logit vectors rather than to clip the private logits themselves. The benefit of taking this approach is that large differences in token probabilities are not generally clipped, which prevents the algorithm from synthetically increasing very low probability tokens to higher levels. Instead, tokens are only clipped if their probability substantially differs to the public distribution.

In a next step, the algorithm samples a token from the Top_k+ vocabulary (defined in Section III). This prevents the LLM from sampling non-zero low-likelihood tokens.

The newly sampled token is then appended to the new shared memory so far. This cycle repeats until the complete memory has been created and an $\langle \text{eos} \rangle$ is sampled, or until the memory runs out of private tokens under the given privacy budget.

Privacy guarantee

The privacy guarantee rests on an important assumption about how tasks are assigned to batches. ?, which is the basis for the InvisibleInk algorithm, state it in Assumption 1 as: “*the assignment of a prompt to a batch depends only on the prompt itself, and not on the other prompts*”. Section 4.1 sets out how the shared memory creation satisfies it.

Under that assumption, Theorem 2 of the paper limits the cost of one run in replace-by-null differential privacy as:

$$\rho_{tok} = \frac{C^2}{2B^2\tau^2}, \quad (4.1)$$

where, C stands for the clipping bound, B for the batch size, and τ for the sampling temperature. For a sentence length of T , this implies

$$\rho_{seq} \leq T * \rho_{tok} = \frac{TC^2}{2B^2\tau^2}, \quad (4.2)$$

which converts, for any $\delta \in (0, 1]$, to (ε, δ) -differential privacy with

$$\varepsilon = \rho_{seq} + \sqrt{4\rho_{seq} \log(1/\delta)}. \quad (4.3)$$

Equation 4.3 sets the trade-off between ε and δ , the two parameters setting the privacy budget. Holding ρ_{seq} fixed, a tighter δ has to be paid for with a larger ε . These two terms define the privacy guarantee, which is:

$$\Pr[M(D) \in O] \leq e^\varepsilon \Pr[M(D') \in O] + \delta \quad (4.4)$$

A mechanism is (ε, δ) -differentially private when, for two neighbouring datasets (D and D'), the probability of any outcome O under the differentially private mechanism M , changes by no more than a factor of e^ε , with a δ % failure probability. In other words, δ defines the chance that the probability of outcome O does differ by more than e^ε (?, Definition 1)⁸.

⁸ ε is not itself a probability, and is not confined to $[0, 1]$, e^ε is the probability

Differences between the two uses of the mechanism

Step 1 and step 3 run the same algorithm with two differences. They use different prompt templates, as reproduced in section 5.2. Step 1 is given the task that resulted in the generation of the memory, whereas Step 3 is given the memory items themselves. Furthermore, step 1 adds a parsing stage, in which the generated string is read as a list of separate labels, since a single differentially private release has to yield several labels at once. Step 3 needs no such stage, because its output is the shared memory content directly.

Deviations from the paper

The algorithm is applied as published, but two details of its deployment differ:

- The original ? paper runs one generation model and this thesis runs a later release of the same family, which section 5.1.2 sets out⁹
- Second, while the InvisibleInk algorithm doesn't generate multiple synthetic examples from one batch, the ? did. But while the Amin et al. algorithm resets the newly generated shared memory after each memory ends for as long as the privacy budget is not exhausted, the labels in Step 1 of this thesis are continuously added to the same string. The reason for making this distinction is because there is no use for duplicate labels. If the algorithm were to reset after each label, it could generate multiple copies of the same label, as was the case in the tests6.1.1. The reason why this is not an issue in the original ? paper is because they would have wanted a training set that reflects the likelihood of certain examples. This difference is only relevant for Step 1, as for Step 3 this thesis' methodology only generates a single memory. The choice of concatenation versus reset therefore never had to be made¹⁰.

⁹? uses a different model family: TinyLLaMA 1.1B, LLaMA3.2-1B, and LLaMA3-8

¹⁰If users were to generate multiple shared memories from one bucket, the author would suggest appending, rather than resetting, for the same reason of duplication as in Step 1.

Implementation

This chapter defines all implementation choices that were made to validate the research question and test the hypotheses defined in 2.2.

5.1 Tech stack

5.1.1 Hardware

Computer

The development machine was a MacOS laptop with 32 GB of memory and 994 GB of storage. The storage was important to install and run the WebArena configurations, since they ship as Docker images of four full websites and take up a lot of space. This machine ran the private and shared memory generation, against Apple's MLX runtime and the Metal Performance Shaders (MPS) backend. Both models are used here: Gemma for the differentially private token-by-token generation, and Qwen for the title and description that follow it.

GPUs

The GPUs that are available to us through the school are a cluster of three NVIDIA A30s. These were used to run the agent, and therefore the Think node, across the 812 WebArena tasks. Although these GPUs are not the quickest processing unit available at the time of this publication, it was all that was freely accessible to the author. Since each experimental condition requires multiple complete pass-through of all 812 tasks, paying for the runs on a cloud computing platform at an approximate cost of £50 per run was infeasible. The available GPUs were shared across multiple students, and therefore had to be used efficiently.

This thesis' work is split across the two machines by what each part needs. The agent loop is throughput-bound, since it repeats across 812 tasks and once per experimental condition, and it runs on the cluster. It is the run-throughs which need to be completed multiple times, and which are thus a barrier to completion, whereas the private and shared memory creation only have to be run once. They do not prevent running multiple run-throughs concurrently, and therefore are not as urgent to complete quickly. It is for this reason that the private and shared memory generation is completed on the MacBook, rather than the cluster. Furthermore, the differentially

private path for shared memories is bound by the token-by-token logit vector prediction over multiple memories, which is very time-consuming. Running it on the laptop prevents competing for shared cluster time.

5.1.2 Software

Programming language

The programming language for this entire project is Python. The reason for choosing it was for multiple reasons: first, it is the programming language the author is most well-versed in. Second, Python hosts a mature scientific stack, built on the array-programming foundation of NumPy (?) and including the deep learning framework this thesis depends on for local logit access, PyTorch (?). Both the differentially private mechanism and the agent loop are therefore assembled from maintained, interoperable libraries rather than reimplemented.

Embedding model

The thesis embeds text using a sentence-transformers model, which is a continuation of the BERT model class. BERT is an encoder-only transformer (?), which had to be adapted before it could be used to compare two sentences and measure their similarity. Three properties of the original model stand in the way.

Firstly, BERT is not trained to recognise similarity. Alongside masked token prediction, it is trained on next sentence prediction, which asks how likely it is that one sentence follows another in a document. This is a wholly different relation than the one required here. A sentence that follows another does not need to resemble it in any way, and semantically similar sentences do not necessarily follow each other.

Second, even if next sentence likelihood were the relation of interest, BERT scores a pair by passing both sentences through the model together. Individual sentences therefore cannot be assigned a single reusable embedding vector, and each sentence has to be scored against every other sentence in the corpus. This results in an explosion of compute and time, which rules the approach out for a memory store that is searched on every Think turn.

Lastly, the raw encodings do not separate cleanly under cosine similarity. BERT's embeddings are anisotropic, meaning they occupy a narrow cone of the vector space rather than spreading through it (?), so arbitrary sentences return high similarity scores.

SBERT was created as a solution to these issues (?). It takes the mean of all token vectors to generate a single fixed-size semantic representation of a sentence, and it fine-tunes the underlying BERT on sentence pairs, so that similar sentences are pulled together in the vector space and dissimilar ones pushed apart.

The model used in this thesis, `all-MiniLM-L6-v2`, is a six-layer reduction of the distilled MiniLM transformer (?), fine-tuned in the sentence-transformers framework on roughly 1.17 billion sentence pairs. It is used in two places. First, to retrieve semantically similar memories for the Think prompt based on the agent's task, and second, to assign each memory to one of the differentially private labels in the shared memory pipeline described in Section 4.2.

The model was chosen for its cost, since the retrieval step is called once per Think turn across 812 tasks per pass-through, which means the per-call cost compounds quickly. At 22.7 million parameters and 384 dimensions it runs offline and deterministically. MTEB (?) reports that no single embedding model dominates across task types, so the choice is a trade-off rather than a ranking, and `all-MiniLM-L6-v2` sits at the small and fast end of it. While a larger encoder would likely retrieve more accurately, this thesis compares configurations against each other

under one fixed retrieval setting, so holding retrieval quality constant across conditions matters more than maximising it.

LLM

The LLMs that this thesis uses were chosen with the aim of firstly, meeting the requirements for the use in terms of output, secondly, of being free of charge, thirdly, of producing high-quality output, and fourth, of producing output in a reasonable time-frame. Lastly, this thesis gave preference to models used by either the ReasoningBank or Amin et al. approach, since the aim is to replicate their mechanisms as closely as possible. These objectives had to be traded off against each other for the different use cases.

LLMs were used in the thesis in five parts:

1. **Agent - Thinking node.** The thinking node in the ReAct loop requires an LLM to take the information from the past actions and observations and make a choice on the next action from a subset of pre-defined actions. This step does not require any intermediate computations, and simply requires a single line naming one action call. The choice of model could therefore be focused on a free and quick model.

In a first step, the author checked for the model that was used to run the agent in the ReasoningBank paper. Unfortunately, the paper used Gemini-2.5 and Claude 3.7, closed-source, paid models. Since there was no budget for this project, and any low-tier free access would have been exhausted well before 812 tasks completed, this thesis had to take a different approach¹.

For open-source models, Qwen is among the most widely adopted open-weight model families, with its technical reports accumulating several thousand citations (??). It could be downloaded for free, and had different parameter settings to choose from to balance speed and intelligence. The model used for the Think node is Qwen3.5 at 4 billion parameters, quantised to NVFP4 (Ollama tag `qwen3.5:4b-nvfp4`).²

The reason for choosing the 4bn parameter model over the 9bn one, is the speed of execution. The 9bn parameter model was taking overall too long, whereas the 4bn parameter model allowed an average of ~ 5 s per Think step³.

Since the thesis aim was not to assess overall success rates, but a comparative assessment instead, the overall intelligence of the model was not as important. Published success rates for open-weight models at or below 8 billion parameters on the full 812-task benchmark span roughly 2% to 24%, and that spread is driven at least as much by the agent harness, the observation format and the prompting strategy as by the model itself (???). The closest published reference point for the configuration used here is ?, who report the same Qwen3.5 4bn model at 24.1% on the full 812 tasks under the BrowserGym evaluation protocol, without fine-tuning. That figure is a useful target rather than a prediction, since the harness differs from the one built for this thesis. While a low absolute success rate

¹This is an important distinction to the tests run by the ReasoningBank paper (?). It would therefore not be unexpected if the model underperformed in comparison to that paper on the 812 benchmarks.

²Qwen3.5 was released without a technical report. The architecture and training are documented for the preceding generation in ?.

³This is an approximate measurement of one configuration and not evidence about any single factor, because moving to it changed the parameter count and the quantisation format at the same time. It was also measured on the laptop during model selection, whereas the evaluation runs were served from the A30 cluster, so it is the basis on which the 4bn model was chosen and not the per-step cost of the reported runs.

would narrow the base of tasks on which a difference between configurations can show, it would not undermine the comparison, which holds the model fixed and varies only the memory condition.

The model is served locally through an Ollama endpoint, rather than being loaded directly into the runner. The reason for this is that the model is quantised, and its quantisation format was built for a newer generation of NVIDIA cards than the A30s available here. Loading it directly through HuggingFace would only simulate the quantisation on this hardware, so the model would keep the accuracy cost of being compressed without running any faster. That would leave a choice between a simulated version that is no quicker, and dropping the quantised model for the full-size weights. Ollama runs the quantised model as it is meant to be run, and avoids the choice altogether.

Ollama also releases the model from GPU memory once it has been idle for a while, whereas a model loaded into the runner holds that memory for as long as the process is alive. Since the A30s are shared across students, this matters.

The same model and endpoint serve every other non-private call in the system for the reasons outlined above.

2. **Success-or-failure judge.** The model used for the success-or-failure judge within the private memory generation is once more the Qwen model with the Ollama endpoint.
3. **Private memory creation.** Equally, the same Qwen model and Ollama endpoint were used for the private memory generation.
4. **Differential privacy mechanism.** As with the earlier LLM uses, the first aim was to replicate the previous paper as closely as possible. Whereas this was not feasible for the ReasoningBank paper's use of LLMs, the Amin et al. mechanism uses the free Gemma model in its paper. This model is also an open-weights model, which is crucial for this algorithm's use case. The mechanism needs the full next-token logit vector over the vocabulary at every sampling step, for each member of the batch and for the public prompt, and it needs to be able to extend that generation one token at a time. The differentially private path consequently runs on a locally loaded Gemma model, through PyTorch and HuggingFace transformers, where the logits are read directly off the forward pass.

The sole deviation of substance from the ? paper is that it uses Gemma 1.1 2B IT, whereas this thesis uses the more recent Gemma 2 2B IT (?). This choice may affect the hyperparameters, since the values the paper reports were tuned on the earlier models.

Gemma 2 2B IT was chosen over the Gemma 2 2B base model for two reasons. First, it has been instruction-tuned, meaning further trained on tasks phrased as natural-language instructions (?), so inputs must be arranged into the model-specific chat template. Second, in the intermediate experiments reported in Section 6.1.1, Gemma 2 2B IT under the chat template produced lessons that fitted the input memory items better than the base model under raw completion.

5. **Shared memory title and description generation.** The Qwen model is used once more, for the title and description generation of the shared memory. This step no longer requires token logit access, so it can be served through the same Ollama endpoint as the Think node. Its inputs are strictly the differentially private content and the differentially private label, so by the post-processing property of differential privacy the call adds nothing to the privacy account.

Testing configuration

The testing configuration consisted of both the tasks and scoring, as well as the environment. The tasks and scoring were provided by WebArena, while the environment was set by BrowserGym.

WebArena

All evaluation runs are completed on the WebArena gym (?), accessed through the BrowserGym wrapper (?). The WebArena repository provides the evaluation environment, tasks, and scoring for 812 different online tasks:

- **Environment.** The WebArena repository provides the configurations for four websites, which mimic the real-world equivalent of GitLab, Reddit, an online store content management system (CMS), and OneStopShop. These configurations allow the tester to run the agent tests locally on replicas of the websites, removing the risk that the agent’s actions take effect on a live service. It also provides a set of utility tools and knowledge resources that the agent can call, if needed, for further closeness to the real world scenario. These include a calculator, a scratchpad, an offline copy of Wikipedia, and a map, the last of which is itself served as a site and carries its own tasks (?). Because the environment resets to a deterministic initial state, these configurations also ensure that the same tests can be run over an extended duration of time, without fear of change to the website.
- **Tasks.** The repository also provides a list of 812 different online tasks, which the agent is given to perform. These tasks are instantiated from 241 templates and are phrased as high-level, realistic natural language commands rather than click-by-click instructions, so the agent has to decompose them itself (?). The example task “*What is the top-1 best-selling brand in Quarter 1 2022*” names an outcome and leaves the route to it open.
- **Evaluation.** Lastly, the repository also provides a program, which validates the functional correctness of each task by checking the outcome rather than the action sequence taken to reach it. BrowserGym surfaces that verdict as the episode reward, and the runner records it when the episode ends, so a task counts as successful when its final reward is greater than zero.

The reason for choosing the WebArena environment was to keep as close as possible to the original testing environment used by ?. The thesis therefore chose to test its hypotheses on the same test set as that paper. Firstly, this ensures that the ReasoningBank framework is useful on this dataset, and that a difference in agent behaviour with memories should be observable. Second, it allows the results of this thesis to be compared to those in the paper.

BrowserGym

While WebArena provides the benchmark tasks and evaluations, BrowserGym provides the testing environment to run the thesis on. It launches and takes action on a Chromium browser using Playwright. While the work for this thesis included creating its own Playwright nodes and Chromium actions, BrowserGym simplifies the work in three ways. First, it exposes the WebArena suite through a single environment call, including the per-task setup and the reward signal that the self-made path did not have access to. Second, it offers a substantially larger action vocabulary than the seven actions the self-made implementation supported, set out in Section 5.2. Third, it catches exceptions raised by an action and returns the error text in the next observation (?), so the failure is fed back to the LLM rather than lost.

It also returns structured observations, in one of three formats. BrowserGym automatically returns all three observation spaces which WebArena has been tailored to (??), which includes the HTML DOM tree, a screenshot, or the accessibility tree of the website URL and content.

Which of the three this thesis feeds into the Think node, and the reason for that choice, is set out in Section 5.2.

5.2 Design choices

This section lays out how the system architecture and algorithm from the previous sections were implemented to assess this thesis’ research question. While Section 5.1 sets out which tools and models the thesis uses, this section sets out how they were configured.

The hypotheses listed in Section 2.2.1 require the system architecture described above to be run on the 812 WebArena tasks⁴ with different configurations. To assess the benefit of shared memories, the thesis requires a pass-through of the following configurations:

- **A - No memory use.** The agent runs without private or shared memories. The trajectories from this run are used to generate private and shared memories for future pass-throughs.
- **B - Private memory use only.** In this pass-through all memories from the previous 812 trajectories are considered to have been created under a single user. This next pass-through is considered to be done from the same user, who therefore has access to all memories from the past pass-through, including the memory from the exact trajectory’s pass-through. The agent thus has access to all private memories using a top-3 retrieval.

Because a memory is retrieved on the embedding of the task it was distilled from, and that task is the identical string to the one being solved, a task’s own memory ranks first on every retrieval. This pass-through therefore measures the ceiling of what memory injection can contribute, and not the transfer between different tasks.

- **C - Shared memories only.** Using the private memories generated under pass-through A, the shared memory mechanism generates the new shared memories. The agent has access to the shared memories only, and none of the private memories for the 812 tasks.
- **D - Private and shared memories.** This last pass-through is used to compare how private and shared memories are used when both are available, and whether the shared memories are helpful or detrimental to the overall functioning of the system. This is a complete implementation of the architecture described in section 4.1.

Each of these pass-throughs requires the agent component of the system. The private and shared memory creation components run once, on the trajectories produced by pass-through A, and their outputs are what pass-throughs B, C and D consume. Next, this thesis describes the implementation choices for each step.

5.2.1 Agent

While WebArena provides the set-up, the ReAct Loop is the engine underlying the agent completing the tasks. The implementation of these is described in more detail below.

⁴This thesis will refer to a pass-through as a complete evaluation of all 812 WebArena tasks under one configuration.

ReAct Loop

The ReAct Loop is run in line with the academic description provided in 2.1.3. The agent loops through observation-thinking-action steps in order to solve the task it has been given:

- The **observation** step is handled by BrowserGym and the WebArena configuration. Although BrowserGym returns an accessibility tree, the HTML DOM, and a screenshot of the website url and content, this thesis only feeds the accessibility tree observation into the LLM’s thought process. The reason for choosing the accessibility tree over the HTML DOM or screenshot representation is that it contains the structure of the page at the lowest cost to the context window, and in a form the action grammar can consume directly. The accessibility tree is a subset of the DOM restricted to the elements that are relevant and useful for displaying a page’s content, so it retains that structure at a fraction of the DOM’s length (?). Its compactness is highly relevant for the class of model used here, since the thesis uses lower-capability models, and ? find on WorkArena that accessibility-tree observations raise the success rate of lower-capability open-source models. In comparison, HTML only helps the higher-capability proprietary ones, because the longer input drives up hallucination in the weaker models. The tree also labels each element with an identifier that the BrowserGym action grammar takes directly, so the LLM can name a click target without grounding it in pixel coordinates, which a screenshot would require of a vision model this thesis does not use. A final implementation choice that this thesis made, is to cap the rendered tree at 12,000 characters, since even the accessibility tree can outgrow the context window on a dense page. The truncation is undertaken from the end and marked with an explicit truncation comment, so that page content never displaces the system prompt or the task.
- The **think** process is handled by the LLM. Each call is made at temperature 0, with the model’s reasoning trace disabled and the reply capped at 64 tokens, which is the length of a single action line. Holding the sampling settings fixed keeps the Think step deterministic given its prompt, so any change in success rate between evaluation runs is attributable to the memories rather than to sampling noise.

The reasoning trace is disabled because the reply grammar has no room for it. Qwen emits its reasoning as a block before its answer, whereas the system prompt asks for exactly one line holding a single function call, so a trace would either have to be stripped before parsing or would be rejected outright and spend a step. The two settings are coupled: a reasoning block would exhaust the 64-token budget before the model reached the action. Disabling it also bounds the cost of a pass-through, since the Think step is called up to 30 times per task, across 812 tasks, once per configuration. While this keeps each pass-through within the time available on a shared cluster, it also removes the surface on which a retrieved memory could act before an action is chosen, which is a limitation this thesis returns to in Chapter 7.1.

The Think module is provided the following information, as part of the system and user prompt:

- **System Prompt.** The initial set of instructions for each thinking step is consistent across each agent loop, and consists of three separate parts.
 - * Firstly, it informs the LLM that it is a web agent controlling a browser through BrowserGym WebArena actions.

- * Second, it fixes the reply format, which is exactly one line holding a single Python function call and no commentary, with element identifiers taken from the accessibility tree.
- * Third, it lists all the possible actions (see table 5.1) that the LLM can return, with one worked example each. This list is generated directly from the BrowserGym action subset rather than written by hand.
- **User Prompt.** The user prompt passes all other instructions that are relevant to the agent. These define the problem, rather than the LLM’s own system.
 - * **Task.** The first item in the user prompt states the agent’s task, which is a copy of the task provided by WebArena, read verbatim when the environment is reset.
 - * **Memory.** Not all evaluation runs include memories. Specifically, the baseline evaluation does not include any memories, so that the thesis has a clear understanding of how successful the agent can be based on the defined ReAct framework, the prompts, the libraries and the models. Later evaluations then append the memories. These are the key addition to the evaluation and should provide helpful context to the LLM for improving its functioning. They are appended as individual line items with both the memory’s title and content, prefixed by the memory-injection instruction reproduced verbatim from ? so that prompt wording is held fixed as a control variable.
 - * **Agent trajectory.** The agent also receives a history of past steps, thoughts, and outcomes from this trajectory. The outcome of each step is either `ok`, if the step was completed without an error message, or the specific failure, which is the error returned by BrowserGym, the exception the step raised, or the parse error the reply produced. These are the working memory which enable it to learn from previous actions, and work towards its goal.
 - * **Accessibility tree.** As discussed above, the LLM is passed the website observation in the form of an accessibility tree.
 - * **Instruction.** The end of the user prompt is a single sentence: *“Reply with exactly one BrowserGym action line.”* This pushes the LLM to respond with a single action item that can be parsed for the action step.
- The **action** space is defined by WebArena once more, and includes actions across three categories. The first includes element operations including clicking, hovering, and typing, while the second enables tab-related actions. Lastly, the third allows URL navigation actions. The action space as published is listed in table 5.1 below. The set actually exposed to the LLM is BrowserGym’s `webarena` subset, which departs from the published table in three places. It adds `select_option` for dropdown elements. It replaces the paper’s `stop(answer)` with two explicit termination actions, `send_msg_to_user` for returning an answer and `report_infeasible` for declaring the task unachievable. And it omits `noop`. Both the grammar printed into the system prompt and the parser that validates the LLM’s reply are generated from this one subset, so the actions the LLM is offered and the actions the environment accepts cannot drift apart.

The agent continues looping through the ReAct framework until the environment terminates, which happens when the LLM emits `send_msg_to_user` or `report_infeasible`. Meeting the goal and declaring the task infeasible are both specified as possible actions for the LLM to take as part of its Think process, so ending the episode is a choice the model makes rather than a state the harness infers.

Action Type	Description
<code>noop</code>	Do nothing
<code>click(elem)</code>	Click at an element
<code>hover(elem)</code>	Hover on an element
<code>type(elem, text)</code>	Type to an element
<code>press(key_comb)</code>	Press a key comb
<code>scroll(dir)</code>	Scroll up and down
<code>tab_focus(index)</code>	focus on i -th tab
<code>new_tab</code>	Open a new tab
<code>tab_close</code>	Close current tab
<code>go_back</code>	Visit the last URL
<code>go_forward</code>	Undo <code>go_back</code>
<code>goto(URL)</code>	Go to URL

Table 5.1: Action Space of WebArena

In addition, to prevent continuous looping, this thesis’ implementation also added a stopping rule after 5 successive identical replies from the LLM, and a max 30 step rule for each task. Five identical replies is the shape that both observed failure modes take, whether a reply the parser keeps rejecting or a click repeated on a page whose state never advances. A parse failure on its own does not end the episode, since the rejected reply and its error are appended to the history and the loop returns to the observation step, so the LLM sees its own malformed output on the next turn and can correct it. While these rules may reduce the agent’s overall capability, they bound the time necessary for each evaluation of all 812 tasks, which has to be repeated once per experimental condition.

Lastly, each completed task writes one row recording the task intent, the full trajectory, the final observation, the agent’s returned answer and the harness reward. Counting the rows whose reward is greater than zero gives the number of successful and failed trajectories for that evaluation run.

5.2.2 Private memory creation

As described above, the private memory creation mechanism first assesses whether a trajectory was successful or failed, and only in a second step creates the memories from this trajectory.

Success-or-failure assessment

The failure-or-success assessment is an LLM-as-judge call, and is separate from the reward check that WebArena performs. Both verdicts are recorded for every trajectory, but the WebArena assessment is what the evaluation in chapter 7 scores. The judge is based on the following prompt from the ReasoningBank paper, shown in table 5.2.

The four insertions into this system prompt are the user intent, the trajectory, the final state, and the bot response:

- The user intent - i.e. the task description -, and the trajectory are both inserted into the judge’s system prompt from the trajectory record.
- The third addition into the prompt is the final state of the webpage, which isn’t part of the trajectory record. The final state is inserted as an accessibility tree of the webpage as the agent last saw it, truncated to the same 12,000-character budget.

Table 5.2: System instruction and input prompt template used by the LLM judge to evaluate web-navigation agent trajectories.

System Instruction
You are an expert in evaluating the performance of a web navigation agent. The agent is designed to help a human user navigate a website to complete a task. Given the user's intent, the agent's action history, the final state of the webpage, and the agent's response to the user, your goal is to decide whether the agent's execution is successful or not. There are three types of tasks: 1. Information seeking: The user wants to obtain certain information from the webpage, such as the information of a product, reviews, map info, comparison of map routes, etc. The bot's response must contain the information the user wants, or explicitly state that the information is not available. Otherwise, e.g. the bot encounters an exception and respond with the error content, the task is considered a failure. Besides, be careful about the sufficiency of the agent's actions. For example, when asked to list the top-searched items in a shop, the agent should order the items by the number of searches, and then return the top items. If the ordering action is missing, the task is likely to fail. 2. Site navigation: The user wants to navigate to a specific page. Carefully examine the bot's action history and the final state of the webpage to determine whether the bot successfully completes the task. No need to consider the bot's response. 3. Content modification: The user wants to modify the content of a webpage or configuration. Carefully examine the bot's action history and the final state of the webpage to determine whether the bot successfully completes the task. No need to consider the bot's response. *IMPORTANT* Format your response into two lines as shown below: Thoughts: <your thoughts and reasoning process> Status: "success" or "failure"
Input Prompt
User Intent: {intent} Trajectory: {trajectory} The detailed final state of the webpage: ``md {cap} `` Bot response to the user: {response if response else "N/A"}

- Lastly, the bot response is the LLM’s last action. If the action was the `send_msg_to_user` action, then the message itself is attached as the response, whereas any other action would result in the literal “N/A” as the bot response.

One failure mode of the extraction is handled explicitly. A judge reply that carries no parseable “*Status*” line is treated as a failure, so a trajectory whose success the LLM did not confirm is never routed to the success prompt.

Memory creation

After marking a trajectory as either successful or failed, a separate instantiation of the LLM then creates up to three summary memories using the prompts set out in the figure below 5.1. Both prompts are reproduced verbatim from ?. An extraction that yields no complete memory item, either because the reply carried no item header or because every item was missing one of the three fields, causes the trajectory to be dropped rather than stored as an empty record.

The extractor call runs at temperature 1.0, reproduced from ?, rather than at the temperature 0 used everywhere else. The memories are therefore drawn stochastically, so a second extraction pass over the same trajectories would not reproduce them exactly. Holding the setting fixed to the published value keeps the extraction comparable to ReasoningBank, at the cost of making the memory set itself a sampled artefact rather than a deterministic function of pass-through A.

The three memories returned by the LLM, which each include a title, description and content, are bundled together with the user ID, the success/failure judgement, and the initial task. Each task is then embedded with the sentence transformer described in section 5.1.2, producing a 384-dimensional dense vector, and added to the bundle. The embedding is computed once, when the record is written, and stored inside the record, so reloading the store never re-embeds. At retrieval time only the incoming task is embedded, and it is compared against every stored vector by cosine similarity. As stated previously, the aim is to ensure the private memory creation resembles the ReasoningBank architecture as closely as possible for comparability, and ReasoningBank likewise embeds the task string and retrieves by cosine search.

5.2.3 Shared memory creation

The implementation questions around the shared memory creation regarded the privacy hyperparameter choice, the privacy budget, the parameters in the four steps of the shared memory creation, and the prompt choice.

Hyperparameters. The privacy hyperparameters are held identical across Step 1 and Step 3, so that the two rounds are directly comparable and their budgets add without further analysis. The values in Table 5.3 are the settings of the pilot configuration and are for a theoretical, rather than practical proof-of-concept. The ϵ in particular is set far above any privacy-meaningful level.

Privacy budget. The privacy budget is not spent directly. For a given target ϵ and batch size s , the code solves for the number of private tokens r that fit inside that budget. Generation then stops once r private tokens have been sampled. Because ϵ is accounted from the budgeted r rather than from the tokens the generation actually consumed, a memory that terminates early is reported at the full cost of its budget. The reported ϵ is therefore an upper bound on what was spent. Two further conditions halt generation before r is exhausted: a total cap of 256 tokens across the private and public branches combined, since this seems sufficiently long for a memory

System Instruction	System Instruction
You are an expert in web navigation. You will be given a user query, the corresponding trajectory that represents how an agent successfully accomplished the task. ## Guidelines You need to extract and summarize useful insights in the format of memory items based on the agent's successful trajectory. The goal of summarized memory items is to be helpful and generalizable for future similar tasks. ## Important notes – You must first think why the trajectory is successful, and then summarize the insights. – You have learned or seen something new from the successful trajectory. – You can extract <i>at most 3</i> memory items from the trajectory. – You must not repeat similar or overlapping items. – Do not mention specific websites, queries, or string contents, but rather focus on the generalizable insights. ## Output Format Your output must strictly follow the Markdown format shown below: <pre> ''' # Memory Item i ## Title <the title of the memory item> ## Description <one sentence summary of the memory item> ## Content <1-3 sentences describing the insights learned to successfully accomplishing the task> ''' </pre>	You are an expert in web navigation. You will be given a user query, the corresponding trajectory that represents how an agent attempted to resolve the task but failed. ## Guidelines You need to extract and summarize useful insights in the format of memory items based on the agent's failed trajectory. The goal of summarized memory items is to be helpful and generalizable for future similar tasks. ## Important notes – You must first reflect and think why the trajectory failed, and then summarize what lessons you have learned or strategies to prevent the failure in the future. – You can extract <i>at most 3</i> memory items. – You must not repeat similar or overlapping items. – Do not mention specific websites, queries, or string contents, but rather focus on the generalizable insights. ## Output Format Your output must strictly follow the Markdown format shown below: <pre> ''' # Memory Item i ## Title <the title of the memory item> ## Description <one sentence summary of the memory item> ## Content <1-3 sentences describing the insights learned to successfully accomplishing the task> ''' </pre>
Input Prompt	Input Prompt
Query: <user query> Trajectory: <trajectory that completes the query>	Query: <user query> Trajectory: <trajectory that completes the query>

Figure 5.1: System instructions for extracting memory items from agent trajectories: the left panel targets successful trajectories (summarizing why they succeed), while the right targets failed trajectories (reflecting on failure and deriving lessons).

Symbol	Meaning	Pilot value
c	Logit clipping bound	50.0
τ	Sampling temperature, private branch	1.0
τ_{public}	Sampling temperature, public branch	1.5
σ	Laplace noise scale for the threshold	0.1
θ	Public/private distance threshold	0.0
r_{max}	Cap on private tokens per output	80
ε	Privacy budget per round	200.0
δ	Privacy δ per round	$1/s$

Table 5.3: **Interim.** Differential privacy hyperparameters of the pilot configuration, applied identically to Step 1 and Step 3. These values are provisional and are superseded once the budget is retuned.

and caps the generation duration, and the model emitting either its end-of-sequence token or the end-of-turn token of the instruction-tuned chat template.

Step 1 - Labels per cycle. The number of labels generated per cycle is 80. The choice of creating 80 labels is a simple deduction that for a batch size of 30, where each task yields 3 memories to a batch, one would need approximately 10 tasks per label. The batch size of 30 is what then determines the privacy budget.

Step 1 - Prompt. The prompt in Table 5.4 is the prompt used in the differentially private mechanism in Step 1 to generate the labels.

Step 1 - Output parsing. The output of the mechanism is a JSON array of label strings, which is parsed with three successive attempts. The output is first parsed as JSON directly. Failing that, a closing bracket is appended, since differentially private sampling can cut the output short before the model reaches it. If this still fails, every double-quoted substring in the output is taken as a label candidate. Any slot that none of the three attempts fills is padded with a placeholder label, and flagged as a fallback in the audit record. A placeholder label will rarely attract memories in Step 2, and the memories that would have been assigned to it are carried into the next cycle rather than lost.

Step 2 - Minimum number of memories for generation. The threshold B , the minimum number of memories a label must attract before a shared memory is generated from it, is set to 5. Every memory in a bucket that does not reach five is written to a carry-over file and re-enters the mechanism at the next cycle.

While the calculation of the actual ρ_{seq} could be recalculated for each shared memory based on the actual batch size B , the batch size is held constant across runs at the minimum number of memories B a bucket must hold before it passes to the content generation. The privacy guarantee is therefore always an upper bound.

Step 3 - Number of memories generated per label. The mechanism produces exactly one shared memory per qualifying label per cycle. While the Amin et al. paper runs Algorithm 1 repeatedly to build up a dataset of many synthetic examples from the same private corpus, this thesis focuses on the simplest implementation with one memory. The number of shared

memories available to the agent grows across cycles instead, as new private memories accumulate and new labels are synthesised.

Step 4 - Title and description generation. The title and description are generated in a single Qwen call at temperature 0. The reply is rejected if the title is absent, runs beyond ten words, or ends in punctuation, and if the description is absent. On rejection the call is repeated once, and if the second reply also fails validation the title and description are derived from the content itself: the title becomes its first eight words, and the description its first sentence. The fallback is deterministic and takes its input from the content released in Step 3, so it is a post-processing step like the LLM call it replaces and costs no additional budget.

Retrieval of the shared memories. Once a cycle has written its shared memories, they become available to the agent alongside its own. The final evaluation run over the 812 tasks retrieves from both stores, so that each agent sees its own private memories and the shared memories the mechanism produced. A second run retrieves from the shared store alone. That run is diagnostic rather than a condition of either hypothesis: it separates a gain that comes from the shared tier from one that comes from the agent's own history, which the combined run on its own cannot distinguish.

System prompts. All system prompts for the shared memory creation are included in the tables below.

Table 5.4: Step 1 - System prompt used by the Gemma model to generate the differentially private labels for each cycle.

System Prompt
Return exactly k labels that summarise the memory items below. Each label must be 14 words and on a DISTINCT facet no two labels may be synonyms, paraphrases, or describe the same aspect. Return only a JSON array of strings. No explanations, numbering, markdown, or extra text.
Example: ["pricing risk", "customer churn", "supply delay", "budget pressure", "staff training", "data quality"]
Memory items: {items}

Table 5.5: Step 3 - System prompt used by the Gemma model to generate the differentially private content for each label bucket.

System Prompt
You will be given memory items distilled from one web-navigation trajectory related to label. Produce a single short paragraph (1 to 3 sentences) that captures the most generalisable lesson from these memory items. Output only the lesson text no title, no header, no markdown formatting, no list, no explanation of what you are doing. Memory items: {items} Lesson:

Table 5.6: Step 4 - System prompt used by the Qwen model to generate a title and description for each synthesised shared-memory item.

System Prompt
You will be given the content field of a shared memory item synthesised from one web-navigation topic (label: {label}). Write a title (1 to 10 words, capitalised, no trailing punctuation) and a one-sentence description that together identify what the content advises. Do not introduce any facts not present in the content. Output exactly two lines in this format and nothing else: Title: <title> Description: <description>

Unanticipated difficulties and critical reflections

This chapter sets out the biggest difficulties the author faced while working on this project, the pivots she undertook, and the critical reflections on them.

6.1 Unanticipated difficulties

6.1.1 Generating usable text with the Amin et al. mechanism

Once the mechanism of ? had been replicated, it proved very hard to generate meaningful text at all. Run on toy examples, the output was gibberish, and partially in other languages including Chinese and Korean. The examples in Figure 6.2 are representative of what the mechanism produced at this stage.

The tests that follow were run on the first instantiation of the mechanism, which generates the round-1 labels, rather than on the round-3 shared-memory content. The round-3 content could equally have served, but a label is one to four words long, which simplified spotting faulty labels at a glance with respect to a bad paragraph. Table 6.1 holds the ten short event reviews used as the private corpus throughout this section, unless stated otherwise. They were chosen for their simplicity and because a human reader can see immediately which label groups them correctly.

Six possible causes were assessed in turn.

- **The base model rather than the instruction-tuned one.** The first model implemented was the Gemma 2 2B base model, which produced answers that were simply non-sensical. ? use Gemma 1.1 2B IT, so it was not clear which model of the newer generation would do best. The author therefore wrote a short script that put the two side by side on the same prompt, holding the clipping bound at $c = 20$, the token cap at 80, and the sampler fixed, and varying only the model and its prompt format¹. The instruction-tuned model was much the more succinct and appropriate of the two when asked to synthesise a summary memory from the toy data in Table 6.1, as Figure 6.1 shows. Given that it has been fine-tuned on chat conversations, this was not a surprise. The 9B and 27B models were also available, but Appendix A.2 of ? reports 75.7%, 76.4% and 76.9% for Gemma 2 at 2B, 9B and 27B on four-shot DBPedia classification, so a thirteen-fold increase in parameters

¹The author used the chat template for the instruction-tuned model and the raw completion for the base model.

id	text
1	I hated the event, terrible.
2	I had such a good time, incredible music.
3	I really enjoyed talking to everyone, really nice people.
4	Whenever I turned around, I had a new person to talk to, it was great.
5	Loved the food
6	Nice weather, good people
7	Never been there before and would give it a solid 8/10
8	Good spaghetti at the end
9	Thought I'd hate it, but I actually didn't!
10	Would go again

Table 6.1: The ten toy memories (`scripts/amin_et_al/examples.csv`) used as the private corpus for the diagnostics in Section 6.1.1.

```

=====
Model: google/gemma-2-2b-it
Regime: chat template

DP-generated lesson (raw):
'The event was enjoyable and engaging, resulting in widespread positive sentiment. \n'

=====
Model: google/gemma-2-2b
Regime: raw completion

DP-generated lesson (raw):
'\nIt was great. I didn't put too much effort into it because I really didn't care. I just went to the event because it was a free and I had an hour or so to spare. I found the experience disappointing. But I also appreciated the very limited time. In hindsight, I was probably over-analyzing it as much as I over-analyzed my research'
```

Figure 6.1: Comparison of Gemma 2 and Gemma 2 IT model

buys around one point. The author therefore stayed with the smaller and quicker model, `google/gemma-2-2b-it`.

- **An unoptimised label prompt.** The prompt had to be adapted rather than borrowed. The template in ? asks the model for another example like the ones it has been shown, whereas round 1 of this thesis asks for a label under which a set of memories can be grouped. Five variants were compared, differing only in the instruction prose, with the private corpus, the parser and the private-token budget held fixed. Table 6.2 gives the labels each returned. From a subjective review, `json_discriminative` was the best of the five: it grouped the data in Table 6.1 most accurately and produced the most distinct labels. It was promoted into the final pipeline as the production `LABEL_PROMPT`, shown in Table 5.4.²

The comparison also shows what the anti-redundancy clause is doing. Asking for distinct facets on its own (`json_discriminative`) yields three usable labels, whereas adding the requirement that one of them must summarise the batch (`json_discriminative_one`) breaks the JSON format entirely, and every slot falls back to a placeholder.

- **Generating the labels in one sequence rather than one at a time.** Algorithm 1 of ? resets the token sequence to empty before each new synthetic example, so every example

²This check ran at a target ε of 200.0 and $\delta = 0.01$. The private-token budget r hit its cap of 80 before the target was reached, so the realised budget was $(\varepsilon, \delta) = (26.2, 0.01)$. The ten toy examples were duplicated ten times each, giving a batch of 100, so that r would stretch to a label of reasonable length.

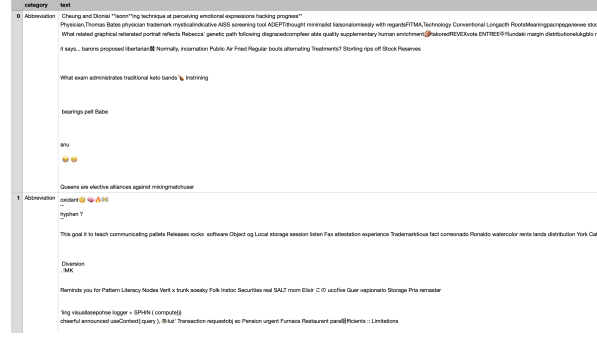


Figure 6.2: View of first two memories generated with the Amin et Al. mechanism using the TREC dataset.

is generated independently from the same batch of prompts and nothing generated earlier enters the context of what follows. This thesis departs from that. It asks for all k labels inside a single JSON array, so each label is conditioned on the ones already emitted. The two were compared directly, holding the corpus, the hyperparameters and the private-token budget fixed. Table 6.3 gives the result. The paper-faithful sequential mode produced the same label twice, whereas the concatenated mode produced three distinct ones, and the deviation was kept.

The intuition as to why concatenation helps is that each memory pushes the next token towards its own preferred direction. Once a token has been committed to the array, the memories that wanted it no longer have a strong preference for any particular continuation, while the memories that have not yet been served still do. Their spike in the logits can then overpower the flat average of generically likely tokens, which carry no context. Generating each label from an empty sequence removes that mechanism, and the batch simply converges on its single strongest preference every time.³

- **An implementation error.** None of the changes above produced useful labels at a defensible privacy budget. Every check reported so far ran at a realised ε of 26.2, which is far outside anything meaningful in practice, whereas ? report usable synthetic text at $\varepsilon = 1$ with δ set to the inverse of the sensitive dataset’s size. Since the paper’s implementation is closed-source, the only way to test whether the fault lay in this thesis’ code was to reproduce the paper’s own experiment. The author therefore ran the mechanism on TREC, one of the classification datasets in the paper, at the paper’s published hyperparameters: batch size $s = 500$, clipping bound $c = 10$, private temperature $\tau = 1.5$, SVT threshold $\theta = 0.3$ and noise scale $\sigma = 0.1$, at $\varepsilon = 1$ and $\delta = 1/n$, which solved to a budget of 31 private tokens. The synthetic questions this produced were not usable either, as Figure 6.2 shows. The failure therefore reproduces on the paper’s own dataset at the paper’s own settings, which points away from a defect specific to this thesis’ code and towards the mechanism’s behaviour at the scale available here.
- **Too tight a clipping bound.** A fifth consideration was that clipping might be squashing the per-memory distributions together, so that the sampled token reflected whatever was generically likely rather than anything the prompt had asked for. To assess this, the author compared the token logits before and after clipping. Figure 6.3 shows that even at $c = 10$ the highest-ranked tokens stay comparatively high. This is not evident from the overall

³This check ran at the same realised $(\varepsilon, \delta) = (26.2, 0.01)$ and on the same duplicated batch of 100 as the prompt comparison above.

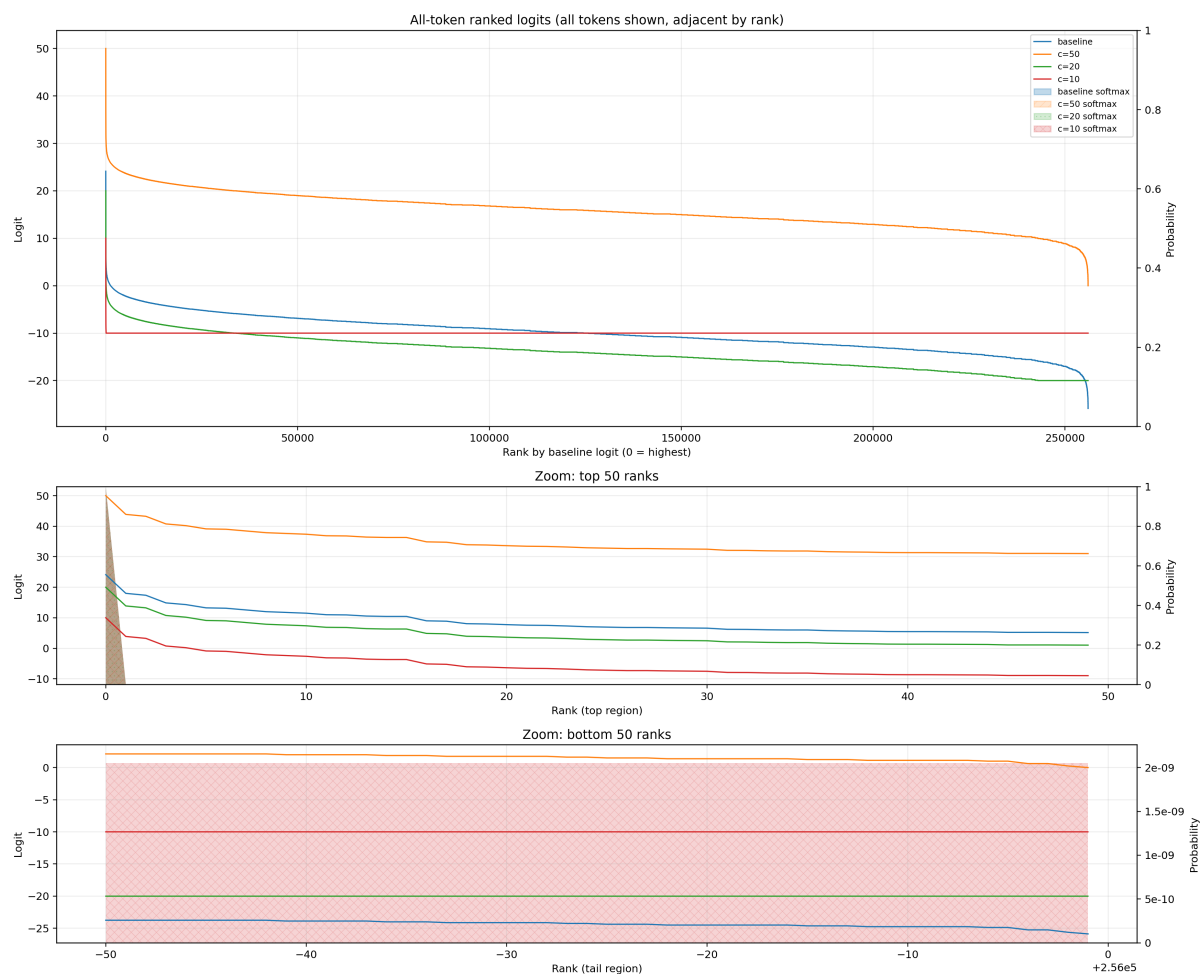


Figure 6.3: Token logit heights

chart, but the zoom makes it clear: the clipping used by ? leaves the top of the distribution where it is and lifts only the bottom, so the top-ranking preferences of each memory survive. Clipping is therefore not what degrades the output.

A second diagnostic on the same question is more informative about what is going wrong. For each of the ten toy memories, the author extracted the private logit vector at the second generated token, clipped it at $c = 50, 20$ and 10 , and took the head of the ranked curve above its largest early drop. The head is small, between one and twelve tokens, and it is identical for the same memory at all three bounds. This means the same memory returns the same top token whether it is clipped at 10 or at 50 . What the heads do not do is agree with one another. The ten top tokens are “negative”, “music”, “positive”, “New”, “positive”, “positive”, “Positive”, “delicious”, “Unexpected” and “positive” five variants of one word and five unrelated ones. Averaging over a batch can only preserve a token that several members support, so with heads this narrow and this divergent there is little for the average to carry. That is a property of the corpus rather than of the clipping bound, and it is the same argument that motivates grouping memories by label before synthesis in Section 4.1.

- **Checking with the author.** The previous bullet led the author to believe there may be a systematic failure that she missed. As such, she reached out to the primary author of the ? paper. This is where she gained the insight needed to produce sensible text for the shared

agents. The feedback from the author was multiple:

claude: generate a table comapring these two results. In particular, compare the epsilon bduget used with the feasible r, i.e. show that the one only allowed for $r=6$, while the other one had not even used half its budget. Second, highlight how invisible ink has a much cleaner text. python scripts/amin_{et}al/modifed_{aminet}al.pyInvisibleInkharnessonscripts/amin_{et}al/10, $T = 80$, $\tau = 1.0$, $top_k = 100$ targetepsilon = 10.0, $\delta = 1e-05$ calibratedC = 2.111099budgetedrho(T) = 1.782696, epsilon(T) = 10.0000Loadingcheckpointshards : 100

Synthetic text: 'The most important lesson is to ensure that the materials and activities relevant to your personal goals are appropriately integrated and displayed throughout. '

Privacy account: tokens generated = 27 / 80 mean $|V_k + | = 127.3$ budgetedepsilon(T) = 10.0000spentepsilon(len) = 5.2606budgetedrho(T) = 1.782696spentrho(len) = 0.601660(venv)(b Pro - 6 : 25 - 26_CE901 - _{SU}_CE902 - _{SP}_{frizlen}_yasmineYasmineFrizlen python scripts/amin_{et}al/WP2₃.pyDelta : 0.1Privacyassessment : targetepsilon = 10.0, $\delta = 0.1$, $s = 10$, $c = 50.0$, $\tau = 1.0$, $\sigma = 0.5$ - $r = 0$ Budgettootighttosampleevenoneprivatetok Pro - 6 : 25 - 26_CE901 - _{SU}_CE902 - _{SP}_{frizlen}_yasmineYasmineFrizlen python scripts/amin_{et}al/WP2₃.pyDelta : 0.1Privacyassessment : targetepsilon = 10.0, $\delta = 0.1$, $s = 10$, $c = 10.0$, $\tau = 1.0$, $\sigma = 0.5$ - $r = 6$

Amin delta validity (Theorem 1 + Appendix C): $\delta_{c,hosen} = 1.000e-01$ $\delta_{m,in}(\text{Theorem1}) = 1.191e-021/n(\text{AppendixC}, n = 10) = 1.000e-01$ $in(0, 1] = \text{True}$ meetsTheorem1tight = True withinAppendixCbound = True valid(allthree) = True Loadingcheckpointshards : 100

Synthetic text: 'The psychology section was shortened in the course of half a day, yet the'

Privacy account: realised epsilon = 9.1414 rho = 3.4800 private tokens used = 6 / 6 public tokens used = 9

- **Replacing the mechanism for the bucketing step.** The last consideration was to keep the mechanism of ? for the shared-memory synthesis alone and to bucket the memories by some other method. Five methods were compared on the same toy memories at the same $k = 4$:
 - The mechanism with its privacy machinery stripped out
 - The differential privacy mechanism as deployed
 - Spherical k -means over the project's own sentence embeddings
 - BERTopic with its dimensionality reduction disabled and its clusterer replaced by the same k -means
 - BERTopic at its own defaults

Table 6.4 gives the labels each produced. The two conventional clustering methods partition the corpus, but neither names its partitions in a way a retrieval step could use: k -means can only return the memory nearest each centroid, and BERTopic's c-TF-IDF returns stopwords. Only the two differential privacy mechanism variants produce labels that read as topics. Set against that, and against the simplicity the mechanism offers in accounting for the privacy budget, the author decided to keep it for both steps.

claude: update the table with teh actual cluster results in scripts/amin_{et}al/outputs/cluster_comparison_extensiv

6.1.2 Hosting the WebArena benchmark

The evaluation could either run on a cloud provider’s servers or on the author’s own laptop. The WebArena repository recommends AWS, and the reason for the recommendation is likely the sheer size of the configurations, which ship as Docker images of full websites. Following that recommendation would have required, according to the documentation, an EC2 instance of at least `t3a.xlarge` with 1 TB of storage, at an approximate cost of £50 for a single pass-through of the 812 tasks. The project was not funded, the design calls for several pass-throughs, and the £100 of student credit available would not have covered them.

The evaluation therefore had to run locally, and the laptop, although fairly empty, did not have the storage to download the configurations and boot the Docker images. After the available space had been maximised, the remaining solution was to borrow a 2 TB external disk from the author’s supervisor, which was sufficient and allowed the project to move forward.

6.1.3 A low success rate on WebArena

The first pass-through, without memories, succeeded on 59 of the 812 tasks, a success rate of 7.27%. This is remarkably low. The largest single reason is the agent model. ? run ReasoningBank on Gemini-2.5-flash, Gemini-2.5-pro and Claude-3.7-sonnet, whereas the constraints set out in Section 5.1 put this thesis on Qwen3.5 at 4 billion parameters. Even against that reference, though, the gap is large: ? report the same model at 24.1% on the full 812 tasks without fine-tuning. The author therefore looked for defects in the harness rather than accepting the number.

claude: add in table that addresses ending, failure percentage and count only (not answer-scored and state-scored). <table> Ending Count Stuck detector fired: 5 identical replies in a row 359 47.7201 158 Killed by the unauthorised-URL safeguard (agent left the instance) 166 22.030 136 Declared the task infeasible via report_{infeasible}749.83143Exhaustedthe30 – stepbudget618.11645Answeredviasend_{msg}to_{user}, judgedwrong597.83920Browserornavigationerror
/table >

Thinking mode. The clearest failure rate was coming from aborting due to being stuck in a repeated action cycle of more than 5 repeated actions in a row. This points towards an inability of thinking intelligently enough. It was therefore the first item to address. The author therefore turned on thinking mode, and increased the thinking cap from 256 to 1024 tokens, since thinking requires more tokens in a response. Furthermore, she added a parsing for any possible <think> tags in the output.

URL translation. With the original run, the agent attempted to access a real website, usually the real URLs of the mirrored websites rather than the localhost copy, 166 times, which is a substantial portion of the failures across the 812 tasks. The solution was therefore to claude: insert solution in clear, and easy way based on my code

- **The evaluator’s LLM judge.** WebArena scores a task with up to three evaluators, and one of them, `string_match`, compares the agent’s answer against a reference. Where the reference is marked as a fuzzy match, the comparison is delegated to an LLM judge, which the harness hard-codes to OpenAI’s `gpt-4-1106-preview`. 118 of the 812 tasks route through that judge: 82 through `llm_fuzzy_match` against a reference answer, and 36 through `llm_ua_match` against the note attached to a deliberately unachievable task. Without an API key those tasks cannot be scored, and the author added her key for the OpenAI assessments. **[INTERIM: re-run success rate to be inserted.]**

- **An action grammar that never asked for the answer.** The system prompt gave the agent the BrowserGym action set but did not tell it to call `send_msg_to_user` once it believed the task was done. 335 of the 812 tasks are scored on the answer string, and on those an agent that solves the task but never reports its answer scores zero.⁴ Table 6.5 breaks the 753 failed trajectories down by how they ended, and the vast majority never issued the action at all.
- **Resetting the database between tasks.** Although the first implementation reset the Docker environment between each task, the author decided not to reset the database after each task, which may have made alterations to the database in terms of posts, CMS data, placed orders, and others. However, since the official WebArena website recommends resetting the database only after the 812 tasks have been completed, and since a single reset is expected to take between 200 - 500 seconds, which would amount to 45 - 113 hours to run a single run-through. claude: add reference: https://github.com/web-arena-x/webarena/blob/main/environment_docker/README.md#environment-reset
- **Memory truncation.** In the original no-memory and private-memory-only run-throughs, truncation hit 371 out of 812 times without memories and 455 out of 811 times with private memories.

Two readings of that first row are consistent with the data. Either the agent was not capable enough and looped or stalled until the 30-step cap terminated it, or it considered the task complete and simply never reported. The corrected prompt was intended to separate them. **[INTERIM: results of the corrected prompt to be inserted.]**

6.1.4 Memory injection breaking the action grammar

The pass-through with private memories returned 57 successes against the no-memory baseline's 59. Read on its own, that number says memory injection made the agent slightly worse. It does not. Paired across all 812 tasks, 18 tasks went from success to failure and 16 went the other way, and an exact McNemar test on those 34 discordant pairs gives $p = 0.86$. The harness's own run-to-run variation is wider than the difference being measured, so there is no effect here to explain.

What the flat headline conceals is a defect in the memory arm that is neither small nor within noise. Across the two runs, the share of Think steps whose reply the action parser rejected rose from 114 of 7,950 in the baseline, or 1.4%, to 3,273 of 8,534 with memories, or 38.4%. 2,358 of those rejected replies open with a Markdown code fence, against 14 in the baseline, and 2,135 of them would have parsed and executed unchanged had the fence been stripped.

The cause is a contradiction between two instructions the agent receives in the same prompt. The system prompt asks for exactly one line holding a single function call and no commentary. The memory-injection instruction, reproduced verbatim from ?, asks the agent to first explicitly discuss whether it wants to use each memory item and then take action. The memory block is appended after the system prompt and before the observation, so it sits closer to the model's recent context, and the model resolves the conflict by wrapping its reply in a fenced block. The parser rejects any reply it cannot read as a single call, and the run continues one wasted step poorer. The stuck detector then compounds it: a rejected reply is written into the trajectory as the step's thought, and five identical fenced replies trip the abort, which is why stuck terminations rose from 362 to 525.

⁴The remaining 477 tasks are scored on the final URL or on the page state, and can succeed without an answer string. The prompt therefore does not affect them.

The damage lands precisely where it is most costly. On 111 tasks the agent produced a valid terminal `send_msg_to_user` or `report_infeasible` inside a fence and the parser discarded it, against 4 such tasks in the baseline. Seven of those 111 are tasks the no-memory pass-through had solved. The honest reading is therefore not that memories failed to help, but that the memory arm as run cannot answer the question, because a formatting defect gates it before the memories reach the action. The fix is small strip the fence in the parser, or drop the discussion clause from the injected instruction and the pass-through has to be repeated before the comparison means anything.

There is a second lesson in this, beyond the bug. The instruction was reproduced verbatim precisely so that prompt wording would stay fixed as an experimental control, as set out in Section 5.2.2. That decision was right in isolation and wrong in combination: an instruction written for an agent that reasons in the open cannot be dropped unchanged into an agent whose reply grammar is a single line and whose reasoning trace has been disabled.

6.1.5 Revising the hypotheses and experimental scenarios

The hypotheses and experimental scenarios set out in the proposal are not the ones this thesis reports against. The reason for the switch is two-fold. Firstly, a simpler design proves the concept more cleanly than one that is closer to reality but more convoluted, and research results should be easy to read. Second, the original design would have required far more pass-throughs, each of which takes several hours; between the time available and the number of blockers described above, it was not feasible.

The substantive change is that the 812 tasks are no longer treated as belonging to separate users accruing shared memories within a single pass-through. Instead, the same 812 tasks are run twice: once to generate the memories, and once to use them. Tables 6.6 and 6.7 set the two designs side by side.

The obvious cost of the revision is that the evaluations no longer speak to whether a shared memory generalises beyond the trajectories it was distilled from. A shared memory is now always applied to tasks the earlier pass-through has already seen, so it may well generalise, but nothing here tests it. What the revised design does show is narrower and still worth having: where users have taken an action on this system in the past, a new user can benefit from what was learnt, through the shared tier.

6.2 Critical reflections

This thesis has illustrated to the author that AI research is never finished. There is always an additional angle from which to see the problem, and always another parameter worth tweaking. The real work of an AI researcher, from this author's perspective, is therefore prioritising between a long list of possible fixes.

The testing reported in this chapter was not empirical. Each comparison was judged subjectively, on a small fixture, by one reader. Testing the prompts, then the shared memories they produce, then the 812 agent trajectories that consume them would have given far more robust results, but the time did not allow for it. Subjective assessment was the stand-in, and it was still preferred over not testing at all.

The defect described in Section 6.1.4 is the sharper version of the same lesson. It was found only because a headline number that looked like a small negative effect was checked against the trajectories underneath it, and the check showed both that the effect was noise and that a much larger problem sat beneath it. Where a subjective judgement on ten toy memories is the best a

schedule allows, the number it produces has to be treated as provisional, and the first thing to do with any result is to ask what it would look like if the harness, rather than the hypothesis, were what the data was measuring.

Variant	In- struc- tion, where it dif- fers	Labels returned
json_array	Re- turn ex- actly k la- bels.	expert choice; delighted staff; creative minds
json_one_array	...one of which needs to sum- marise the mem- ory items be- low.	Pleasant experience; Good quality; Happy memory
json_discriminative	...that sum- marise the mem- ory items be- low. Each label ...on a DIS- TINCT facet no two la- bels may be syn- onyms, para- phrases, or de- scribe	Positive Experience; Good Feedback; Pleasant Interactions

Mode	Label 1	Label 2	Label 3
Concatenated array (this thesis)	Positive experience	Pleasant experience	Good quality
Sequential, one label per sequence (?)	Positive Experience	Great experience	Great experience

Table 6.3: Labels produced by the two generation modes on the toy fixture at a realised $(\varepsilon, \delta) = (26.2, 0.01)$, $k = 3$, $r = 80$.

Method	Labels	Cluster sizes
Mechanism, as deployed using InvisibleInk	Customer journey; Travel experiences; Personal interests; Recommendation	69, 75, 0, 0
Mechanism, amin et al privacy stripped	Personal Interests; Contact Information; Job & Education; Personal Experiences	69, 3, 10, 18
Mechanism, amin et al with privacy	personal interests; topical discussions; occupational details; future plans	14, 76, 4, 6
Spherical k -means	nearest memory to each centroid, truncated to four words	19, 41, 9, 31
BERTopic, k -means clusterer	you do my to; to you the it; the it you of; to you the that	41, 31, 19, 9
BERTopic, own defaults	you to my do; it you the to; outliers (HDB-SCAN)	25, 10, 65

Table 6.4: Six bucketing methods on the same 100 memories at $k = 4$. The mechanism as deployed ran at a realised ε of 10.9 with $\delta = 0.01$ and $r = 80$. BERTopic at its own defaults chose three clusters rather than four and assigned 65 of the 100 memories to an outlier bucket.

Ending	Count	% of failures
Terminated without <code>send_msg_to_user</code> (ran out, stuck, or errored mid-task)	620	82.3%
<code>report_infeasible(...)</code>	74	9.8%
<code>send_msg_to_user(...)</code> but wrong	59	7.8%
Total	753	100%

Table 6.5: How the 753 failed trajectories of the no-memory pass-through ended.

Proposal	This thesis
H1 Improvement over baseline. Does the shared memory tier improve agent task success rate over a private-memory-only baseline?	H1. A shared memory tier distilled from other users' trajectories raises task success rate above a no-memory baseline. The claim fails if the cross-user condition does not statistically improve on the no-memory condition.
H2 Privacy guarantee. Does the ? privacy mechanism, applied to the generalisation pipeline, preserve a meaningful fraction of that improvement at a given (ϵ, δ) ?	H2. That gain survives the privacy mechanism. At $\epsilon \leq 5$, success rate under the differentially private shared tier sits closer to the unrestricted-sharing condition than to the no-memory condition. The claim fails if the private tier recovers less than half the distance between them.

Table 6.6: The hypotheses as proposed and as revised. The revision changes the baseline of H1 from private-memory-only to no-memory, and replaces “a meaningful fraction” in H2 with a stated budget and an explicit failure condition.

Proposal	This thesis
A No memory baseline. Each task executed by an independent agent with no memory store. Lower bound: no within-user learning, no cross-user sharing.	A No memory use. The agent runs without private or shared memories. Its trajectories generate the memories all later pass-throughs consume.
B Unrestricted memory. Ten agents execute the 812 tasks assigned at random, accumulating episodic memories throughout, each with access to every user's private memories and to all shared memories. Upper bound if privacy were not a concern.	B Private memory use only. All memories from pass-through A are treated as one user's. The agent retrieves the top three private memories, including the memory distilled from the identical task.
C Cross-user learning. Ten simulated users spread across the 190 task clusters, at most one task per cluster each, so no user meets the same task type twice. Any gain over A is cross-user transfer.	C Shared memories only. The shared mechanism runs over the private memories from pass-through A. The agent retrieves from the shared store alone.
D Within-user learning. Ten simulated users, each assigned whole clusters, so no cross-user sharing occurs. Any gain over A is within-user learning from repeated exposure.	D Private and shared memories. The agent retrieves from both stores. This is the complete architecture of Section 4.1.

Table 6.7: The experimental scenarios as proposed and as revised. The proposal simulated ten users inside one pass-through; the revision runs the same 812 tasks twice and varies which memory store the second run may read.

Results and Evaluation

* Success rate of model with private memories versus without memories versus with shared memories only versus with both private and shared memories

7.0.1 Table of results

7.0.2 Agreement between the LLM judge and the WebArena harness

The LLM judge of section 5.2.2 and WebArena’s programmatic reward assign an outcome to every trajectory independently of one another. Table 7.1 cross-tabulates the two over the 812 tasks of the baseline run.

The two verdicts agree on 725 of the 812 trajectories, or 89.3%. The disagreement is asymmetric: the judge marks a trajectory successful that the harness scored as a failure 78 times, against 9 trajectories in the opposite direction. The judge therefore over-reports success, which is the direction the design of the pipeline makes it most sensitive to.

This asymmetry does not affect the reported success rate, since the harness reward is what the evaluation scores. It affects the memories instead. The judge verdict is what routes a trajectory to one of the two extraction prompts, so those 78 trajectories were distilled with the prompt that asks what made the trajectory work, on trajectories that did not work. A separate 4 trajectories produced no parseable memory item at all and were dropped.

7.1 Discussion

	Harness success	Harness failure	Total
Judge successful	50	78	128
Judge failed	9	675	684
Total	59	753	812

Table 7.1: **Interim.** Agreement between the LLM-as-judge verdict and the WebArena harness reward across the 812 baseline trajectories.

Conclusion and future work

8.1 Conclusion

8.2 Lessons learnt

8.3 Limitations and future work

The work in this thesis was bounded by a few months of research, which meant the research questions and the approach had to be scoped tightly enough to reach a finished result within that time. Limitations were accepted as a consequence, and several work streams that would have strengthened the thesis were set aside rather than pursued.

8.3.1 Limitations

There are a number of limitations to this methodology. Each of them is outlined below:

- **Privacy accounting.** The privacy guarantee of the mechanism deployed here rests on the proofs given by ?. Step 1 and Step 3 are each accounted for correctly under those proofs, but the accounting does not extend to the threshold that decides which buckets pass from Step 2 into Step 3. That threshold is computed from the raw memories rather than from a differentially private output. Because the label set released by Step 1 is public, the decision the threshold makes is visible. An observer who sees which labels produced a shared memory learns, for every label, whether at least x private memories were assigned to it. Publication certifies that a bucket reached the threshold, and the absence of a shared memory signals that it did not. The gate is deterministic, so at the boundary a single memory decides whether a label yields any output at all.

The simplest way to remove the leak is to remove the decision. If a shared memory is generated for every label irrespective of how many memories the label attracted, then nothing about the private memories determines which outputs appear, and the accounting closes over Steps 1 and 3 alone. While this costs nothing in privacy, it is paid for in utility, as a sparsely populated bucket yields a memory dominated by the mechanism's noise

rather than by the lesson it is meant to carry and pollutes the database of useful shared memories.

- **Heterogeneity in trajectories.** The tasks that produced the trajectories, and therefore the private and the shared memories, are the same tasks on which the shared memories are then evaluated. For the private-memory condition this means the agent's context contains a memory distilled from an earlier attempt at the identical task, which is a far more favourable case than a deployment would offer, where a user is unlikely to phrase a task in exactly the way it was phrased before. The private-memory condition is therefore an inflated comparator, and the bar it sets for the shared memories is correspondingly high.

The direction of the effect on the shared memories themselves is less clear. They are generated from the same set of examples, so they are correlated with the evaluation tasks and may benefit from the same inflation. On the other hand, the value of a shared memory lies in the higher-level lesson it abstracts from several trajectories, and such a lesson is most useful precisely where the new task is not a repetition of an old one, which this evaluation cannot reward. Separating how far shared memories transfer to unseen tasks from how far their benefit is a replication of tasks already seen would be a worthwhile direction for future research.

- **Limitations of differential privacy.** Differential privacy bounds the additional disclosure caused by a single record's participation. It does not promise that nothing about a person can be learned from the release, since a conclusion that follows from a population-level regularity holds with almost the same probability whether or not that person contributed (? , §2.3.2). The stronger requirement, that nothing learnable from a release be learnable without it, cannot be met by any release that is useful, once an adversary is allowed arbitrary side information (?).

This matters here because the records being aggregated are not independent. Colleagues within one organisation run overlapping tasks and know each other's roles, so one employee's memories carry information about another's. Under such correlation the formal guarantee still holds exactly as stated, but the protection an employee would intuitively expect is weaker than ϵ alone suggests, because an adversary who models the dependence between colleagues can sharpen inferences about a specific person beyond what the per-record bound appears to allow (?).

- **Practical limitations.** Deploying this system in a firm would require two decisions that this thesis leaves open. The first is where the Step 2 threshold sits. This thesis chose the threshold purely based on a privacy-utility trade-off, whereas in a firm it also determines how many colleagues must independently have met a situation before the lesson drawn from it is circulated, which is an organisational judgement rather than a tuning parameter.

The second is how far the release should reach. The threshold counts memories, and the mechanism admits at most one memory per user into a batch, so a bucket that passes carries contributions from that many distinct people. It does not record which parts of the organisation those people sit in. A bucket assembled entirely from the legal department clears the same gate as one drawn from several different teams, but the first produces a shared memory that may be considered private information from any other department but the legal one. A possible solution would be to install a provenance rule that restricts a shared memory to the narrowest organisational unit its contributors span

8.3.2 Future work

Each limitation above points to work that would follow directly from it. The directions below are independent of them, and would be worth taking on their own terms:

- **Division of private memories.** The current shared memory generation in Step 3 only calls an LLM once on an agent’s private memories from one task. Further work could assess the trade-off between the batch-size increase when attaching each private memory to a separate LLM call, versus the LLM’s potential to identify more pertinent memories among the three ReasoningBank memories when including all three in one prompt.
- **Different memory generation mechanisms.** The private memories here are created with the ReasoningBank framework. Replacing it would test how far the sharing mechanism depends on the form of the memory it is given. Alternative frameworks such as ? and ? may provide additional insights into what form of memory is best shareable across users, and where this mechanism breaks down.
- **Alternative website observation mechanism.** The agent observes each page as an accessibility tree. Comparing this against screenshots or raw HTML as the observation in the ReAct loop would show whether the benefit of the shared memories depends on the observation format, or whether the lessons they carry are about the task rather than about how the page is read.
- **Improved memory retrieval.** Memories are retrieved here by exact cosine similarity over the task embeddings, which was chosen for its simplicity. Retrieval quality is set by what part of the memory is embedded, its representation, and the scoring. The directions worth testing are therefore what a memory is labelled with as an embedding value, the embedding model, and the retrieval mechanism.
- **Semantic and procedural memory.** This thesis limited itself on generating a framework defined for episodic memory only. Further research could assess how to adjust the mechanism for semantic and procedural memory, possibly even budgeting privacy across the different memory tiers.
- **User-level differential privacy.** The methodology rests on example-level differential privacy, which is only elevated to user-level differential privacy under the assumption that users contribute only one example to the set of shared memories. However, over the course of a lifetime, any user will likely contribute multiple memories, which would generate memories across different labels. Since the joint output over a user’s contributions may reveal additional information about him, this no longer follows this thesis’ privacy accounting. Extending the algorithm to a guarantee that is user-level in that stronger sense is left to future work.

This is not an exhaustive list, but a first set of directions the author considers worth taking.

Annex

Figure 9.1: "Decision-curator" command

```
---
name: decision-curator
description: Extract durable decisions from recent commits and
  ↳ reconcile them into .claude/memory/decisions.md. Invoked
  ↳ automatically after `git push` via a hook.
---

# Decision curator

Runs in headless mode after a push or heavy work. Keeps
`.claude/memory/decisions.md` accurate, terse, current.

## Configuration

Maximum commit window per run: 50. If the window is larger,
curate only the most recent 50 and write a warning to the log.

Schema version: v1 (must match the marker in decisions.md;
if it does not match, exit 0 and log a schema mismatch).

## What counts as a decision

Include: architectural choices, library or dependency choices,
API or data contracts, naming conventions, explicit behavioural
rules ("we always do X when Y"), security or privacy constraints,
deployment or environment commitments.

Exclude: implementation steps, refactors that do not change
behaviour, bug fixes that do not reveal a new constraint,
formatting changes, work-in-progress notes, deliberation about
why a decision was hard.
```

If a commit shows only code changes with no underlying decision, record nothing for it.

Ask the user for reasoning where it is not clear. Do NOT make any
 ↪ assumptions around reasoning.

Procedure

1. Check current branch with ``git rev-parse --abbrev-ref HEAD``.
2. Read ``.claude/memory/.last-curated-sha`` ``LAST_SHA``. If the file is missing or empty, log "no marker" and exit 0.
3. Verify ``LAST_SHA`` still exists in current history:
``git cat-file -e LAST_SHA 2>/dev/null``. If it does not exist (force-push or rebase rewrote it), fall back to the merge-base with ``origin/<primary-branch>~50`` or the oldest reachable commit, whichever is more recent. Log the fallback explicitly.
4. Run ``git log --pretty=format:"%H%n%s%n%b%n---END---" LAST_SHA..HEAD``.
 ↪ If empty, exit 0 with a "no commits" log entry. Do not move the marker.
5. Read the existing ``.claude/memory/decisions.md`` in full. Verify the schema marker is ``v1``; if not, log mismatch and exit 0 without changes.
6. From the commit window, extract candidate decisions using the inclusion rules above. Classify each under one of the existing sections.
7. For each candidate, check the log for an entry it supersedes or contradicts. If found, REPLACE the old entry. If not, APPEND under the correct section. Never keep both.
8. Phrase every entry as one declarative sentence stating the current decision. No history. No reasoning. No "we decided" just the decision. Example: "Sessions are stored in Redis with a 24-hour TTL." Not: "After considering Postgres, we moved sessions to Redis."
9. Remove "(none yet)" placeholders from sections that now have entries.
10. Read through any agent history for design choices and
 ↪ reasoning, and add any relevant information.
10. Write the updated file back.
11. Update ``.claude/memory/.last-curated-sha`` to current ``HEAD``.
12. Append one line to ``.claude/memory/curator.log`` in the form:
``<ISO timestamp> | curated <LAST_SHA>..<HEAD_SHA> on <branch>``
 ↪ `` | added=N replaced=M scanned=K``

Hard rules

- Never delete the file's section headers or the schema marker.
- Never write speculation, todos, or open questions.
- Never reference commit SHAs or PR numbers inside the log file itself.
- If you find a decision in the commit window that contradicts something in ``.CLAUDE.md`` (not the decision log), STOP, do not

- modify anything, log `conflict with CLAUDE.md, manual review needed`, and exit 0.
- On any unexpected error: do not modify `decisions.md` or the marker. Log the error with a stack trace and exit 0.

Figure 9.2: "Thesis-task-curator" command

```

---
name: thesis-tasks-curator
description: Headless post-push/MR scanner that maps recent commits
  → to thesis tasks and appends dated progress notes. Never changes
  → status brackets. Invoked automatically by hooks do not invoke
  → manually unless asked.
---

# Thesis tasks curator

Runs headlessly after a `git push` (incremental mode) or `gh pr
  → create`
(branch-wide mode). Appends progress notes to
  → `.claude/thesis/tasks.md`
based on commit content. Never changes status brackets.

## Configuration

Primary branches: `main`, `master`, `develop`. Only runs when `HEAD`
  → is
on one of these. Override by setting env var `CURATOR_BRANCHES` to a
comma-separated list.

Schema version: v1 (must match the marker in tasks.md; if it does not
match, exit 0 and log a schema mismatch).

## Trigger modes

**Incremental** (invoked with "incremental mode"):
- Scans commits since `.claude/thesis/.last-curated-sha`
- If marker is missing or empty, scan the last 10 commits and log "no
  marker, using fallback window"
- Updates `.claude/thesis/.last-curated-sha` to HEAD on completion

**Branch-wide** (invoked with "branch-wide mode"):
- Scans all commits on the current branch vs `master`:
  `git log --pretty=format:"%H%n%s%n%b%n---END---" master..HEAD`
- Does not update `.last-curated-sha` (only incremental runs do that)

On any push from a non-primary branch, exit 0 with a "non-primary
branch, skipping" log entry.

## Task keyword map

```

Match commit subjects and bodies against these terms to identify
 → relevant
 tasks. A commit may match multiple tasks.

Task	Match terms
1.1	langgraph, pipeline, scaffold, think node, act node, observe → node, graph
1.2	accessibility tree, playwright, ally, aria, focusable
1.3	action pars, action grammar, click, type command, parse → failure
1.4	multi-step, termination, max step, stop signal, reasoning → trace
1.5	private memory, rag, retrieval, dense retriever, top-k, → memory store
1.6	reasoningbank, memory generation, distillation, llm-as-judge, → outcome classification
2.1	batch assembly, cross-user batch, j memories
2.2	gemma, logit, huggingface, transformers, logit extraction
2.3	private prediction, gaussian noise, public token, amin, dp → mechanism
2.4	shared memory store, shared store, shared memory integration →
2.5	batch assignment, assumption 1, compliance, hash-based, topic → categor
3.1	webarena, docker, reddit, gitlab env, shopping, cms, maps, → smoke test
3.2	browsergym, agent wrapper, observation api, action api
3.3	task assignment, user simulation, intent_template, scenario
3.4	pilot run, 50 tasks, qualitative
4.1	scenario a, no memory baseline, baseline
4.2	scenario b, unrestricted memory
4.3	scenario c, cross-user only
4.4	scenario d, within-user
4.5	privacy-utility, epsilon, , privacy sweep, trade-off
4.6	analysis, reporting, success rate, statistical significance
4.7	sharing off, optional scenario
5.1	methodology chapter
5.2	background, related work, literature
5.3	results chapter
5.4	discussion chapter
5.5	introduction, abstract, conclusion
5.6	proofread, submit, final

Matching is case-insensitive. Partial matches count (e.g. "logit"
 → matches
 "logit extraction"). If a commit matches no terms, record nothing for
 → it.

Procedure

1. Check current branch: ``git rev-parse --abbrev-ref HEAD``. If not

```

    primary, exit 0 with log "non-primary branch, skipping".
2. Determine mode from the invocation prompt ("incremental" vs
   "branch-wide").
3. Collect commits for the mode (see Trigger modes above). If the
   ↪ commit
   window is empty, exit 0 with log "no commits to scan".
4. Read `.claude/thesis/tasks.md` in full. Verify schema marker is
   ↪ `v1`;
   if not, log "schema mismatch" and exit 0.
5. For each commit in the window:
   a. Combine subject + body into one string, lowercase.
   b. Check against the keyword map above.
   c. For each matched task number, record: task number, short SHA
      (first 7 chars), commit subject.
6. For each task that has at least one matched commit, locate the
   ↪ task
   block in tasks.md (by task number, e.g. `**1.1`). Append one line
   immediately after the last existing line of that task's block:
   ...
       Progress (YYYY-MM-DD): <short-sha> "<commit subject>"
   ...

Use today's date (ISO format). Indent to match the task body (2
   ↪ spaces).
If the same commit has already been noted for this task (check for
   ↪ the
   SHA in the existing text), skip it do not duplicate.
7. Write tasks.md back only if at least one note was added.
8. If mode is incremental, update `.claude/thesis/.last-curated-sha`
   ↪ to
   current HEAD SHA.
9. Append one line to `.claude/thesis/curator.log`:
   `<ISO timestamp> | <mode> | <branch> | tasks_noted=N |
   ↪ commits_scanned=K`

```

Hard rules

- Never change a status bracket: ``[ ]``, ``[~]``, ``[x]``, ``[!]``, ``[-]``.
- Never delete any line from tasks.md.
- Never duplicate a progress note (check SHA before writing).
- Never write speculative notes only note commits that are in the
 ↪ git
 log, with their exact subject line.
- If tasks.md does not exist or is unreadable, log the error and exit
 ↪ 0.
- On any unexpected error: do not modify tasks.md or the marker. Log
 ↪ the
 error and exit 0.

Figure 9.3: "Thesis-tasks" skill

```

---
name: thesis-tasks

```


description: Read, update, and reason about the thesis task tracker
 → at `.claude/thesis/tasks.md`. Invoke when I mention the thesis, a
 → work package (WP1-WP5), a specific task number (e.g. 2.3), a
 → milestone (M1/M2/M3), or ask "what's next", "what's blocked", or
 → "where am I on the thesis."

Thesis task tracker skill

The source of truth is `.claude/thesis/tasks.md`. Read it before answering any question about thesis progress.

When invoked

1. Read `.claude/thesis/tasks.md` in full.
2. Verify the schema marker is ``v1``. If not, report mismatch and stop.
3. Answer the user's question grounded only in the file. Do not invent tasks. If they ask about something not in the file, say so and offer to add it.

Updating tasks

When the user reports progress or asks to update a task:

1. Identify the exact task (by number) and the requested status change.
2. Show the user the proposed edit (old bracket new bracket, plus any note line) and ask for confirmation before writing.
3. On confirmation, edit the file with the smallest change that achieves the update. Never reformat unrelated tasks.
4. If a task is being marked done that has open dependencies (e.g. marking 4.1 done while 3.3 is still open), flag the inconsistency before writing.

Adding tasks

Only add tasks when the user explicitly asks. New tasks need a number (next free in the WP), a title, a "Done when" criterion, and notes if relevant. If the user describes a task without these, ask for what's missing.

Hard rules

- Never silently change a status.
- Never delete a completed task.
- Never duplicate proposal content into the file reference proposal section numbers instead.
- If the user asks "what should I work on next", consider dependencies (e.g. 2.5 blocks 4.5; 1.6 blocks 2.1; 3.3 blocks 4.1-4.4), proximity to milestones, and current status markers. Propose 1-3 candidates with reasoning. Do

not pick for them.

Figure 9.4: "Editing academic text" skill

```

---
name: "editing-academic-text"
description: "Edit, refine, review or correct Yasmine's existing
→ academic drafts - thesis chapters, proposal sections, methods and
→ implementation writing, and LaTeX source. Use this skill whenever
→ she hands over a draft and asks to edit, refine, review, clean
→ up, tighten, proofread or assess it, or says \"make this sound
→ like me,\" \"improve this,\" \"is this clear,\" or \"check this
→ section.\" This skill governs editing only: it changes what is
→ wrong, unclear or unsupported in text she has already written,
→ and reports gaps rather than filling them. Do not use it to draft
→ new sections from scratch."
---
```

Editing Academic Text in Yasmine's Voice

This skill governs one job: editing a draft Yasmine has already
→ written, in the academic register. That register covers
→ argumentative academic prose (proposal sections, thesis chapters,
→ related work) and implementation and methods writing
→ (architecture, experiment design, evaluation procedure), which
→ carries an extra evidential standard set out in Pass 2.

The skill does not cover drafting. Where the draft is missing
→ something, the finding is reported, not written.

****The boundary between correcting and writing.**** Three cases, and
→ they are not the same:

- New words **inside** her sentence, required to make a claim true or a
→ clause grammatical, are part of editing. Correcting "5 successive
→ equivalent actions" to "5 successive identical replies" is a
→ correction, not a rewrite.
- New sentences, paragraphs or sections that supply content the draft
→ does not have are out of scope. Write the finding in the change
→ report - what is missing, why a reader needs it, where it belongs
→ - and leave the text alone.
- A missing transition between two of her paragraphs sits between the
→ two. Ask before writing one; do not manufacture a bridge that
→ commits her to a connection she has not made.

The Governing Rule

Her draft is the baseline, and it is better evidence of her voice
→ than any style guide, including this one. She applies to prose
→ the rule she applies to code: change only what needs changing.

****The default is her sentence, not yours.**** A sentence survives
 → untouched unless it is:

1. grammatically broken;
2. ambiguous or awkward enough to stall a reader;
3. factually wrong against the code, the data or the cited source;
4. missing information the sentence itself needs to be intelligible.

"I would have phrased it differently" is not a reason. Replacing a
 → correct sentence with a better one is a failed edit, not a good
 → one, because the cost – her voice diluted, her reasoning restated
 → as someone else's – is paid on every sentence while the benefit
 → is marginal on each.

Three Levels of Intervention

Every finding resolves to exactly one of these. Deciding which is
 → most of the editorial judgement in the job.

Level	What it covers	Action
---	---	---
Fix silently	Misspellings, subject-verb disagreement, missing articles, clauses that collapse mid-sentence, unbalanced LaTeX environments, unmatched braces, duplicate labels, <code>\ref</code> targets that do not resolve	Correct in place; list in the report without commentary
Fix and report	Claims contradicted by the artefact, ambiguity a reader would stall on, overpacked paragraphs, AI-register phrasing, unqualified numbers from uncontrolled comparisons, first person in academic register	Correct inside her sentence, then state what changed and on what evidence
Flag, do not touch	Anything that changes a research claim, hypothesis, comparator, scope boundary or design rationale; missing content; internal inconsistencies in her argument; terminology collisions; prose that conflicts with a table or figure; numbering that clashes with another document	Leave the text as written and raise it in the report, in her terms

Escalation is one-directional: when unsure whether a finding is level
 → 2 or level 3, treat it as level 3. A flagged non-issue costs her
 → one line of reading. A silently rewritten research claim costs
 → her a claim she no longer recognises as hers and may not catch.

What the Editor Is Looking For

Run three passes in order. Assume all three are wanted unless she
 → narrows the scope herself.

Pass 1 – Language, structure and red thread

Read for the reader, not for the rulebook. The symptoms:

- ****A word that can be removed without losing the point.**** Remove it.
 - This is the test for filler, not a general licence to shorten.
- ****A paragraph making more than two or three moves.**** The most
 - common offender states a choice, a constraint, a rejected
 - alternative and a solution at once, and reads as a compressed
 - spec rather than prose. Split it at the seam she already wrote
 - in; do not redistribute the content.
- ****A paragraph that could be moved elsewhere without a reader noticing.**** The connection to the paragraph before it is missing.
 - This is a Pass 3 finding, not a Pass 1 fix: report it rather than
 - inventing the link.
- ****A sentence carrying two claims that need separate support.****
 - Splitting is a fix; merging her short sentences is not, since she
 - does not write in fragments and rarely needs joining.
- ****Ambiguity in a technical clause.**** Test by asking whether two
 - readings of the sentence would send a reproducer down different
 - paths. If they would, fix it. If both readings are the same in
 - practice, leave it.
- ****Prose that no longer sounds like her.**** Chiefly first person,
 - hype, generic hedging and the AI tells listed below. This is a
 - fix, and it is high priority, because it is the failure she
 - notices first.

Pass 2 - Every claim checked against the artefact

This pass is the reason implementation and methods writing needs an editor rather than a proofreader.

- ****The repository is ground truth.**** Not the draft, and not the
 - planning documents. Drafts go stale as the code moves; planning
 - docs record intent that has often been superseded. Where the code
 - and a planning document disagree, the code wins and the
 - divergence is reported.
- ****Check what drifts**:** defaults, thresholds, model identifiers,
 - parameter values, quantisation formats, file formats, prompt
 - contents, action sets, control flow, evaluation procedure. Cite
 - ``file:line`` for every verdict in the report.
- ****Read identifiers component by component.**** A model tag encodes
 - family, generation, parameter count, quantisation and packaging
 - as independent facts. Collapsing them is exactly the error to
 - catch.
- ****Recompute every number that appears in the text**** from the
 - underlying data. Never carry a figure across from a summary, an
 - earlier message or a subagent's report without checking it.
- ****Numbers from uncontrolled comparisons must be labelled at the point they appear.**** If a speed-up, improvement or delta could
 - have come from more than one changed variable, name the confound
 - and say plainly whether the controlled test was run. She would
 - rather report a measurement honestly bounded than let it imply a
 - cause it cannot support.

- ****Distinguish the published design from the implemented one.****
 - Where a benchmark or paper defines something and her
 - implementation uses a variant, both belong in the text with the
 - delta named. Neither may be silently presented as the other.
- ****Honour standing exclusions.**** If she has said a behaviour in the
 - code is wrong and will be changed, leave the text describing her
 - intended design and label it as excluded from correction. Carry
 - the exclusion forward for the rest of the work.

Pass 3 - What is missing (report only)

Ask what a reader needs in order to reproduce the work or defend it

- under examination, and what the section claims that nothing in
- the document establishes. Everything found here is written into
- the report, not into the draft.

- ****Undefined symbols.**** Anything appearing in a displayed equation,
 - algorithm box or figure that the text never defines.
- ****Load-bearing justifications.**** These are the ones most often
 - missing: why a design satisfies a formal requirement, which
 - assumption a guarantee rests on, what failure mode a threshold
 - was introduced to prevent. A constraint stated without its
 - failure mode is incomplete, and the reasoning is usually recorded
 - in code comments or decision logs rather than in the draft.
- ****Unpaired descriptions.**** She pairs what a mechanism does with
 - what it does not do. Where a natural limit exists and is
 - unstated, name the gap.
- ****Section boundary errors.**** Architecture sections carry the
 - generalisable design; experiment sections carry this
 - implementation's specific choices; a tech-stack chapter carries
 - model identities and library versions. State the boundary call
 - and the reason rather than moving material silently.
- ****Claims with no concrete anchor.**** Where a claim rests on no
 - example, number, tool or scenario, flag it for her to supply one
 - rather than inventing the example.

What Must Survive the Edit

These are the things a competent general copy-editor would smooth

- away, and each removal makes the text sound less like her. They
- are not errors.

****Her signposting, verbatim.**** Load-bearing and not interchangeable

- with smoother equivalents: "The key problem here is...", "As a
- result, this thesis will...", "Rather than [X], the thesis will
- [Y]", "The purpose of this project is not to [X]. Instead, it
- focuses on [Y].", "As outlined in [ref]", "As shown above", "As
- discussed above", "While [X] provides [Y], [Z] is the...", "This
- section describes each in turn.", "[X] is defined by [Y] once
- more", "In addition, to prevent [problem]...", "Lastly, [the
- source] also provides..."

****Uneven ordinal sequences.**** "Firstly... Second... Third..." and
 → "First... and second...", often closing with "Lastly". Do not
 → regularise into parallelism; the unevenness is hers.

****Concession-then-justification.**** The compact form **While [cost], it*
 → *[benefit]** - "While this may reduce the agent's overall
 → capability, it bounds the time necessary for each evaluation."
 → Never relocate the downside to a later sentence or a footnote,
 → and never cut it.

****Her grammatical colloquialisms.**** Preserve as written: "The reason
 → [X] is because [Y]"; "The reason for choosing [X], is [Y]" with
 → the comma before the verb; sentence-initial "And"; adverb
 → placement a copy-editor would move ("was taking overall too
 → long"); "once more" as a back-reference to a source already
 → introduced.

****Her scaffolding.**** Sectioning and heading capitalisation exactly as
 → written, including inconsistent capitalisation of defined terms.
 → List environments as chosen: an `\itemize` with `\textbf{Label.}`
 → lead-ins does not become `\paragraph{}` headings or running
 → prose, and the reverse also holds. Paragraph boundaries, unless a
 → paragraph is doing too many things.

****Her ordering.**** If she introduces A then B then C, the edited
 → version introduces A then B then C, even where another order
 → would flow better. Reordering is a rewrite.

****Her LaTeX conventions.**** `\texttt{}` for code identifiers, function
 → names, file names and model tags, underscores escaped.
 → `\textit{"quoted string"}` for strings quoted verbatim from a
 → prompt, paper or interface. `\citep` for parenthetical citations,
 → `\citet` when the authors are the sentence subject, `\citeauthor`
 → for a bare name on second mention, `\citep[Lemma 3.17]{key}` when
 → located. Bibliography keys as `authorYYYYkeyword`, lower case.
 → Tables as `\begin{table}[t]` with `\caption` then `\label`. Match
 → the citation command convention already in the document rather
 → than introducing a new one.

What Must Not Survive the Edit

****First person.**** A hard constraint in thesis, proposal and research
 → writing. Replace "I" or "we" with the referent that fits: "this
 → thesis argues" for argumentative claims, "the proposed system"
 → for technical description, "the team" for collaborative work.
 → Vary the substitute rather than defaulting mechanically to one.

****AI register.**** Highest-priority sweep, run before any other quality check. Hard bans: horizontal rules; "In today's [X]"; "In the rapidly evolving"; "It's important to understand that"; "It's worth noting that"; "Here's a breakdown of"; "When it comes to"; "At its core"; "Not only... but also" as flourish; "Moving forward"; "This ensures that" as a paragraph closer; "Overall," as a paragraph opener; "delve into"; "leverage the potential of"; "navigate the complexities of"; "the landscape of"; "a paradigm shift".

Structural tells, which matter more than the banned phrases because they survive a find-and-replace:

- Em dashes used as a default connector. More than one or two per page means most should become commas, full stops or subordinate clauses.
- Tricolons for rhythm rather than content. Three genuine reasons stated as three sentences are fine; "the reason is threefold" as a wind-up is not.
- Predictable paragraph shape: claim, elaboration, wrap-up beginning → "This ensures/enables/allows".
- More than one colon per paragraph, which signals sentences being used as headers.
- Single-sentence pivot paragraphs ("LLMs changed this."). Build the transition into the final clause of the preceding paragraph or the opening subordinate clause of the next.
- "It is not X, it is Y" as a rhetorical pivot, which defines a thing negatively instead of stating what it is.

The test: read the passage aloud. If it could have been written by any model for any user on any topic, it fails regardless of which specific phrases it avoids.

****Filler.**** Adverbs carrying no information ("very", "really", "quite", "highly", "significantly" unless quantified); throat-clearing openings; redundant pairs ("clear and concise", "each and every", "various different"); empty intensifiers.

****Hype and unsupported comparatives.**** "Groundbreaking", "revolutionary", "cutting-edge", "state-of-the-art" unless quoting a source. "Significantly better" or "vastly superior" without data.

****Generic hedging.**** "It is worth noting that", "one might argue that", "it could be argued that". Where she holds a position, the text states it and qualifies it with specifics. Passive hedging over a decision she has already made is a fix, not a preservation.

****Imprecise vocabulary.**** The specific term rather than the
 → approximate one: "dense retrieval" not "semantic proximity",
 → "differential privacy" not "privacy-preserving techniques",
 → "context window" not "memory limitations". Generic references
 → ("recent studies show", "many researchers have found") need a
 → named source or a flag. One word for one thing: where a term
 → already carries a strict technical meaning in the document, a
 → loose reuse for a related idea is a terminology collision and
 → goes in the report.

****Inflated synonyms.**** "Leverage", "utilise", "facilitate" where
 → "use", "help" or "enable" work.

Inline Markers in Her Drafts

- `{claude: ...}` and `{claude reference: ...}` are instructions.
 → Execute them using the material she supplied, then delete the
 → marker and its contents. Never leave one in the deliverable.
- `<justification> ... </justification>` blocks contain research she
 → has already done. Use the content; never let the block's
 → conversational register leak into the prose it informs; delete
 → the block.
- `{YASMINE to check}` and similar are addressed to her, not to
 → Claude, and record a decision only she can make. Do not delete
 → them and do not answer them in the prose. Convert to a comment
 → that will not render, and raise the underlying question in the
 → report.
- `{**SOURCE**}`, `[XXX]` and other placeholders mark honest
 → uncertainty. Never resolve one by fabricating a plausible value.
 → Either verify it against a primary source or leave it and list
 → it.

Verifying Before Overriding Her

She has asked explicitly not to be given assumptions. In editing,
 → that means:

- Check claims against the primary artefact - the code, the paper,
 → the data - not against plausibility or memory.
- Read enough of a source to confirm it says what the citation
 → implies. Prefer peer-reviewed papers and arXiv preprints over
 → blog posts, documentation and secondary summaries, and prefer a
 → source's own statement of a definition, theorem or assumption
 → over any secondary description of it, including the project's own
 → planning documents and code comments.
- Before asserting that a source does **not** contain something,
 → confirm the whole source was retrieved. A truncated fetch reads
 → exactly like an absent passage. State what was checked and what
 → was not.
- Where a `.bib` entry cannot be fully verified, mark the gap rather
 → than inventing the remainder.

- Hold a correction to the same standard as the claim it replaces. A
 - confident correction that turns out to be wrong costs more than
 - the error it was meant to fix.
- Ask clarifying questions before starting rather than after
 - delivering. The recurring unknowns are deliverable format,
 - placeholder policy and section boundaries.

Provisional values stay marked as provisional and carry a visible ``%``
 → `INTERIM`` comment so they cannot be mistaken for final settings.
 → Content belonging to a chapter that does not exist yet is parked
 → under a banner comment naming its destination rather than
 → deleted.

After editing LaTeX, verify mechanically: balanced environments,
 → matched braces, no duplicate labels, and a listed set of
 → cross-references that resolve outside the file so she can confirm
 → them.

Reporting the Edit

Deliver the edited text and, alongside it, a companion notes file
 → grouped by reason:

1. ****Corrected claims**** - as a table, each row carrying the original
 → wording, the correction and the ``file:line`` or citation it rests
 → on.
2. ****Language and structure fixes**** - grouped, not enumerated
 → sentence by sentence.
3. ****Left untouched deliberately**** - and why, particularly where a
 → general copy-editor would have changed it.
4. ****Open decisions for her**** - flagged findings, gaps,
 → inconsistencies, terminology collisions.

Lead with anything that blocks submission. Never bury a correction to
 → a research claim inside a list of typo fixes.

Calibration

Correcting inside her sentence - right

Draft: "a stopping rule after 5 successive equivalent actions"
 Code: the comparison is between successive replies, not actions.
 Edit: "a stopping rule after 5 successive identical replies"

Why it works: the claim is corrected and the sentence shape, position
 → and surrounding prose are untouched. The report cites the file
 → and line where the comparison is implemented.

Correcting inside her sentence - wrong

Edit: "To prevent unproductive loops, the implementation halts an
 ↳ episode once the model returns five consecutive identical
 ↳ responses."

Why it fails: the claim is now right and everything else is gone. Her
 ↳ connective, her cadence and her sentence boundary have been
 ↳ replaced for no reason beyond the editor's preference, and
 ↳ "unproductive loops" quietly substitutes a general category for
 ↳ the specific failure modes she named.

Leaving a sentence alone - right

Draft: "And the reason for choosing Playwright, is because it
 ↳ provides resilient functionality that reduces failures, including
 ↳ an auto-waiting mechanism."

Correct action: leave it. Sentence-initial "And", the comma before
 ↳ the verb and "the reason is because" are all hers. Narrowing
 ↳ "resilient functionality" to the auto-waiting mechanism alone
 ↳ would also misrepresent the scope, since resiliency is broader
 ↳ than that one example. Nothing here stalls a reader or is untrue.

Flagging rather than filling - right

Draft: a paragraph stating that the mechanism satisfies user-level
 ↳ differential privacy, with no statement of which assumption the
 ↳ guarantee rests on.

Correct action: leave the paragraph as written. In the report:
 ↳ "Section 4.2 asserts a user-level DP guarantee. The text does not
 ↳ name the assumption it rests on (per-user contribution bound?),
 ↳ and an examiner will ask. This is content only you can supply -
 ↳ flagging rather than drafting it."

Why it works: the gap is identified precisely and located, and no
 ↳ guarantee she did not make has been invented on her behalf.

Figure 9.5: Coding standards

Coding Standards

Setup

```
```bash
make install
```
```

Installs all production and development dependencies (pytest, black,
 ↳ ruff, mypy, ipython) from `pyproject.toml` and the Chromium
 ↳ binary Playwright drives.

Ollama (required from WP1.3 onwards)

The WP1.3 Think node defaults to a local open-weight model served by
 → [Ollama](https://ollama.com), so an Ollama install and a pulled
 → model are runtime prerequisites for ``scripts/langgraph/WP1_3.py``
 → (and anything downstream that uses
 → ``agent_memories.services.ollama_client.OllamaClient``). ``make`
 → `install`` does ****not**** handle this Ollama is a brew install, not
 → a pip package.

```
```bash
brew install ollama
ollama serve & # start the local server on
→ http://localhost:11434
ollama pull qwen3.5:4b-nvfp4 # project default (MLX-tagged)
```
```

```
```bash
ps aux | grep 'ollama runner --mlx-engine' | grep -v grep # should
→ show the subprocess
curl -s http://localhost:11434/api/tags | grep qwen3.5 # should
→ list qwen3.5:4b-nvfp4
```
```

Sentence-transformers (used from WP1.5 onwards)

The WP1.5 RAG retrieval pipeline depends on a small
 → sentence-transformers encoder (``all-MiniLM-L6-v2``, ~80 MB) bound
 → via the ``Embedder`` class in ``agent_memories.memory.embedder``. The
 → model is fetched from the HuggingFace Hub the first time
 → ``scripts/memories/WP1_5.py`` runs, then cached locally; no extra
 → setup step is needed beyond ``make install`` (which installs
 → ``sentence-transformers`` as a regular dependency).

Remotes

This repo pushes to two places: ****GitLab (cseegit)**** the marked repo
 → and ****GitHub**** a public mirror. ``origin`` is configured to push
 → to both at once.

Expected ``git remote -v``

After a working clone, the output should be:

```
```
github https://github.com/yasminejune/shared_agentic_memory.git
→ (fetch)
```
```

```

github https://github.com/yasminejune/shared_agentic_memory.git
→ (push)
origin git@cseegit.essex.ac.uk:25-26-ce901-su-ce902-sp/25-26_CE901-
→ SU_CE902-SP_frizlen_yasmine.git (fetch)
origin git@cseegit.essex.ac.uk:25-26-ce901-su-ce902-sp/25-26_CE901-
→ SU_CE902-SP_frizlen_yasmine.git (push)
origin https://github.com/yasminejune/shared_agentic_memory.git
→ (push)
...

```

``origin`` has one fetch URL (cseegit) and two push URLs (cseegit +
→ GitHub). Run ``git remote -v`` after every fresh clone to confirm
→ the setup.

Commands

| Command | Pushes to |
|---|--------------------------------|
| <code>`git push origin <branch>`</code> | Both cseegit and GitHub |
| <code>`git push github <branch>`</code> | GitHub only |
| <code>`git fetch origin`</code> | cseegit only |

The default ``git push -u origin <branch>`` updates both remotes in a
→ single command. Use ``git push github <branch>`` only when you want
→ to update the public mirror without touching cseegit.

Why it's set up this way

- **cseegit is the marked repo.** All MRs, the squashed ``master``
→ history, and assessment happen on GitLab.
- **GitHub is a public mirror**, kept in sync so the work is visible
→ outside Essex.
- **Never** repoint ``origin`` at GitHub alone submission relies on
→ cseegit having the latest commit on ``master``.

Re-creating the setup after a fresh clone

A clone from cseegit will only have ``origin`` pointing at cseegit. To
→ restore the dual-push setup:

```

```bash
Add cseegit as an explicit push URL (this overrides the implicit
→ fetch-URL fallback)
git remote set-url --add --push origin git@cseegit.essex.ac.uk:25-26_
→ -ce901-su-ce902-sp/25-26_CE901-SU_CE902-SP_frizlen_yasmine.git

Add GitHub as a second push URL on origin
git remote set-url --add --push origin
→ https://github.com/yasminejune/shared_agentic_memory.git

Add github as a separate remote for GitHub-only pushes

```

```
git remote add github
→ https://github.com/yasminejune/shared_agentic_memory.git

```

Verify with ``git remote -v`` the output should match the block above.

---

## ## Branch Naming

```
...
<ticket_number>_<brieft_description>
...
```

Examples:

```
- `16175_repository_structure`
- `16180_langgraph_agent_memory`
- `16195_evaluation_pipeline`
```

Branch directly from ``master``. One branch per ticket.

```
```bash
git checkout master
git pull origin master
git checkout -b 16180_your_feature_name
git push -u origin 16180_your_feature_name
...
```

Commit Messages

Format: ``[TICKET] Brief description``

For work-in-progress commits during development, a single line is
→ fine:

```
...
[16175] Add shared memory store
[16175] Fix token count in summarise.py
[16175] Add unit tests for memory module
...
```

For squashed (final) commits, add bullet points summarising what was
→ done:

```
...
[16175] Add repository structure

- Add src/ layout with __init__.py files
- Add pyproject.toml with dev dependencies
- Add coding standards and MR template
```

```
...
```

```
---
```

Commit Squashing

Squash all commits on a branch into one before merging. This keeps
 → ``master`` history clean and makes each ticket traceable as a
 → single commit.

Method 1: Interactive Rebase (Recommended)

```
```bash
See how many commits to squash
git log --oneline master..HEAD

Rebase the last N commits (replace N with the count)
git rebase -i HEAD~N
```
```

In the editor that opens:

- Keep the first line as ``pick``
- Change all others to ``squash`` (or ``s``)
- Save and close, then write the final commit message

```
...
```

```
pick alb2c3 [16175] Initial structure
squash e4f5g6 [16175] Add tests
squash i7j8k9 [16175] Fix linting
...
```

Method 2: Soft Reset (Alternative)

```
```bash
Soft reset to master keeps all changes staged
git reset --soft master

Commit everything as one
git commit -m "[16175] Add repository structure

- Add src/ layout with __init__.py files
- Add pyproject.toml with dev dependencies
- Add coding standards and MR template"
```
```

Force Push After Squashing

Squashing rewrites history, so a force push is required to update the
 → remote branch:

```
```bash
git push --force-with-lease origin your-branch-name
```

```
...
```

Only force push to your own feature branches. Never force push to  
 ↳ ``master``.

```

```

## ## Package Structure (``__init__.py``)

Add ``__init__.py`` to every folder your code imports from. Leave it  
 ↳ empty unless you want a clean public API.

### **\*\*Add it to:\*\***

- ``src/<package>/`` and all sub-folders (``config/``, ``services/``,  
 ↳ ``utils/``)
- ``tests/`` and all sub-folders (``unit/``, ``unit/config/``,  
 ↳ ``unit/services/``, ``unit/utils/``)
- ``evaluation/`` and any importable sub-folders

### **\*\*Do not add it to:\*\***

- ``docs/``, ``scripts/``, ``data/`` not imported, just files
- Project root

### **\*\*Only expose a public API if needed:\*\***

```
```python
# services/__init__.py only if you want callers to write `from
↳ services import LLMService`
from .llm_service import LLMService
```
```

```

```

## ## Managing Dependencies

All dependencies live in ``pyproject.toml``. Do not use  
 ↳ ``requirements.txt``.

### **\*\*Adding a new dependency:\*\***

1. Install it: ``pip install <package>``
2. Add to ``pyproject.toml``:
  - ``dependencies`` needed to run the project
  - ``[project.optional-dependencies] dev`` tooling only (linters,  
 ↳ test runners)
3. Reinstall: ``pip install -e ".[dev]"``

### **\*\*Version pinning:\*\***

| Syntax               | When to use                                                     |
|----------------------|-----------------------------------------------------------------|
| <code>`&gt;=`</code> | Default allows updates                                          |
| <code>`~=`</code>    | Minor version stability (e.g. <code>`langgraph~=1.0.9`</code> ) |

---

```
| `==` | Only when exact version is critical |
| no pin | Avoid unpredictable across environments |
```

```

```

## ## Daily Workflow

```
```bash
make format      # auto-fix style with black - checks "is the code
→ formatted correctly?"
make lint        # auto-fix imports and style with ruff - checks
→ "does the code look suspicious or sloppy (imports, names, common
→ bugs)?"
make typecheck   # static type check with mypy - checks "do the
→ types line up with what we claimed?"
make test        # run test suite with coverage report
make ci          # all checks, no auto-fix use before opening an MR
```
```

```

```

## ## Pre-MR Checklist

```
```bash
make ci  # must exit with no errors before opening an MR
```

- [] All tests pass
- [] No linting errors
- [] Type checking passes
- [] New functionality has at least one test
- [] No secrets or credentials in the diff
- [] `pyproject.toml` updated if new dependencies were added
- [] Commits squashed into one (`git log --oneline master..HEAD`
→ should show 1 commit)
- [] Branch is up to date with master (`git fetch origin && git
→ rebase origin/master`)

```

```

```

## ## Merge Request Process

When work is complete, tested, and squashed into a single commit:

1. Go to GitLab **\*\*Merge Requests\*\*** **\*\*New merge request\*\***
2. **\*\*Source branch:\*\*** your feature branch **\*\*Target branch:\*\***
  - `master`
3. **\*\*Title:\*\*** your squashed commit title (e.g. `[16175] Add repository structure`)
4. **\*\*Description:\*\*** copy the bullet points from your squashed commit
  - message
5. Tick **\*\*"Delete source branch when merge request is accepted"\*\***



6. Tick **\*\*"Squash commits when merge request is accepted"\*\*** (safety  
→ net if not already squashed locally)
7. Click **\*\*Create merge request\*\***, then merge when ready

---

## ## Decision memory system

This project maintains an automated log of durable design decisions at ``.claude/memory/decisions.md``. The log is loaded into every Claude Code session via an import in ``.CLAUDE.md``, so any Claude (or human reading the file directly) starts with the project's current decisions in mind.

### ### What goes in the log

Architectural choices, library and dependency choices, API and data contracts, naming conventions, explicit behavioural rules, security and privacy constraints, deployment commitments.

Excluded: implementation details, refactors, bug fixes, work in progress, the deliberation behind a decision.

Each entry is a single declarative sentence stating the current decision. When a new decision supersedes an old one, the old entry is replaced, not appended to. The log is meant to be short.

### ### How entries get added

Two paths:

1. Automatic: after every ``git push`` on a primary branch (``main``, ``master``, ``develop``), a ``PostToolUse`` hook fires the ``decision-curator`` skill in headless mode. It reads commits since the last curated SHA, extracts decisions using the rules above, reconciles them against the existing log, and writes back.
2. Manual: run ``/note-decision <text>`` inside Claude Code to record a decision that did not arise from a commit (meeting outcomes, constraints discovered while debugging, policy choices). The slash command applies the same rules.

Both paths log to ``.claude/memory/curator.log``.

### ### Files

| Path                                            | Purpose                    | Tracked in git? |
|-------------------------------------------------|----------------------------|-----------------|
| <code>`.claude/memory/decisions.md`</code>      | The decision log itself    | yes             |
| <code>`.claude/memory/.last-curated-sha`</code> | Marker for the curator     | no              |
| <code>`.claude/memory/curator.log`</code>       | Run history of both skills | no              |

```
| `.claude/skills/decision-curator/SKILL.md` | The automatic curator
→ | yes |
| `.claude/skills/note-decision/SKILL.md` | The manual slash command
→ | yes |
| `.claude/settings.json` | The hook that triggers the curator | yes
→ |
| `.CLAUDE.md` | Imports the decision log | yes |
```

### ### Operating notes

- The hook only fires when push is run from inside a Claude Code session. Pushes from a plain terminal do not trigger the curator.
- The curator only runs on primary branches. Feature-branch pushes are ignored on purpose.
- The curator runs on Haiku to keep cost and latency bounded.
- On force-push or rebase, the curator falls back to a recent window and logs the fallback. Review the log after any history rewrite.
- If the curator fails, it never modifies ``.decisions.md`` or the marker; it only writes to the log. Check the log if a decision you expected to appear did not.
- Edits to ``.decisions.md`` made by hand are preserved by the curator's supersession logic, but if you restructure the file (rename sections, remove the schema marker), the curator will refuse to run until the schema is restored.
- To rebuild the log from full history, delete ``.claude/memory/.last-curated-sha``, restore it to the SHA you want to curate from (e.g. the first commit), and trigger a push or run ``.claude -p 'Run the decision-curator skill now.'`` manually. Expect the 50-commit window to apply.

---

### ## Thesis task tracker

Forward-looking task list for the MSc thesis lives at ``.claude/thesis/tasks.md`` and is curated via the ``.thesis-tasks`` skill. Separate from the decision memory at ``.claude/memory/decisions.md``: tasks are what is planned, decisions are what is settled.

The tracker is not auto-loaded into every session. Invoke the skill explicitly or by mentioning the thesis, a work package, a task number, or a milestone, and Claude will read the file.

Status markers: ``. [ ]`` open, ``. [~]`` in progress, ``. [x]`` done, ``. [!]`` blocked, ``. [-]`` deferred or cancelled.

To update a task, ask Claude to change its status. The skill will confirm the edit before writing.

### ### Automated progress notes

A second headless skill, ``thesis-tasks-curator``, fires after every ``git push`` (incremental scan) and every ``gh pr create`` (branch-wide scan). It maps commit content to task numbers and appends dated ``Progress (YYYY-MM-DD): <sha> "<subject>"`` lines to matched tasks. It never changes status brackets it only surfaces evidence for the user to act on. Run log: ``.claude/thesis/curator.log``.